



Facultad de Ingeniería  
Escuela de Ingeniería en Computación e Informática

**“CONTROLREPRO: SISTEMA DE GESTIÓN INTELIGENTE PARA LA REPOSICIÓN  
DE PRODUCTOS EN PYMES”**

Proyecto para optar al título de Ingeniería en Computación e Informática

Autor:  
Sebastián Cerda Fuentes  
Profesor tutor:  
Cristian Eduardo Rojas Catalán

Colina, Chile

## ÍNDICE DE CONTENIDOS

|                                                                                                   |           |
|---------------------------------------------------------------------------------------------------|-----------|
| ÍNDICE DE FIGURAS .....                                                                           | 5         |
| ÍNDICE DE TABLAS.....                                                                             | 6         |
| Resumen .....                                                                                     | 7         |
| <b>CAPÍTULO I INTRODUCCIÓN .....</b>                                                              | <b>8</b>  |
| 1. Introducción .....                                                                             | 8         |
| 1.1 Contexto .....                                                                                | 9         |
| 1.2 Definición del problema .....                                                                 | 9         |
| 1.4 OBJETIVOS.....                                                                                | 11        |
| 1.4.1 Objetivo General .....                                                                      | 11        |
| 1.4.2 Objetivos Específicos.....                                                                  | 11        |
| 1.4.3 Matriz de Trazabilidad de Objetivos Matriz de Trazabilidad – Objetivos<br>Funcionales ..... | 12        |
| <b>1.4.4 Calidad de los Objetivos.....</b>                                                        | <b>15</b> |
| A. Criterios metodológicos utilizados .....                                                       | 16        |
| B. Evaluación del Objetivo General .....                                                          | 16        |
| C. Evaluación de Objetivos Específicos .....                                                      | 16        |
| D. Consistencia interna y cobertura del problema .....                                            | 17        |
| E. Viabilidad y riesgos controlados .....                                                         | 18        |
| F. Verificabilidad (métricas y evidencias).....                                                   | 18        |
| <b>1.5 Alcance y Limitaciones.....</b>                                                            | <b>18</b> |
| 1.5.1 Alcance .....                                                                               | 18        |
| 1.5.2 Limitaciones.....                                                                           | 19        |
| <b>CAPÍTULO II FUNDAMENTACION DEL TEMA .....</b>                                                  | <b>20</b> |
| 2. Fundamentación del tema .....                                                                  | 20        |
| 2.1 Solución propuesta .....                                                                      | 21        |
| Componentes principales de la solución .....                                                      | 21        |
| Enfoque tecnológico .....                                                                         | 22        |
| Pertinencia y valor agregado .....                                                                | 23        |
| 2.2 Soluciones de mercado .....                                                                   | 23        |
| 1. Sistemas ERP (Enterprise Resource Planning) .....                                              | 23        |
| 2. Software de Inventario especializado .....                                                     | 23        |
| 3. Soluciones locales y planillas adaptadas .....                                                 | 24        |
| Diferenciación de ControlRepro .....                                                              | 24        |

|                                                          |           |
|----------------------------------------------------------|-----------|
| 2.3 Diseño de alto nivel .....                           | 24        |
| Estructura general del sistema.....                      | 24        |
| Flujos principales del sistema .....                     | 25        |
| Pertinencia del diseño .....                             | 25        |
| 2.4 Factibilidades.....                                  | 27        |
| 2.4.1 Factibilidad Técnica.....                          | 27        |
| 2.4.2 Factibilidad Operacional .....                     | 27        |
| 2.4.3 Factibilidad Económica.....                        | 28        |
| 2.4.4 Factibilidad Legal.....                            | 28        |
| 2.5 Modelo de Vistas 4+1 .....                           | 29        |
| 2.5.1 Vista Lógica.....                                  | 29        |
| 2.5.2 Vista de Despliegue.....                           | 31        |
| 2.5.3 Vista de Procesos .....                            | 31        |
| 2.5.4 Vista Física (Desarrollo) .....                    | 34        |
| 2.5.5 Vista de Escenarios (+1) .....                     | 34        |
| Síntesis del Capítulo II – Fundamentación del tema ..... | 34        |
| <b>CAPÍTULO III MATERIALES Y MÉTODOS .....</b>           | <b>36</b> |
| 3.1 Metodología de gestión del proyecto .....            | 36        |
| 3.2 Metodología de desarrollo de software.....           | 37        |
| Enfoque metodológico adoptado.....                       | 37        |
| Ciclo de desarrollo aplicado.....                        | 37        |
| Beneficios de la metodología .....                       | 38        |
| Ejemplo aplicado al proyecto.....                        | 38        |
| 3.3 Ambiente de desarrollo y gestión.....                | 39        |
| 3.3.1 Herramientas de desarrollo.....                    | 40        |
| 3.3.2 Control de cambios y versiones .....               | 40        |
| 3.3.3 Gestión del proyecto y colaboración .....          | 41        |
| 3.3.4 Beneficios del ambiente configurado.....           | 41        |
| 3.4 Scrum aplicado al proyecto .....                     | 42        |
| 3.4.1 Roles del proyecto.....                            | 42        |
| 3.4.2 Procesos y eventos del proyecto.....               | 43        |
| 3.4.3 Definición del sprint.....                         | 43        |
| 3.4.4 Definición de prioridad .....                      | 43        |
| 3.4.5 Definición de puntos de esfuerzo .....             | 44        |
| 3.4.6 Product Backlog.....                               | 44        |
| <b>3.5 Planificación del proyecto .....</b>              | <b>45</b> |
| 3.5.1 Cronograma macro (12 semanas).....                 | 45        |
| <b>3.5.2 Hitos y entregables .....</b>                   | <b>46</b> |
| 3.5.3 Plan de sprints (detallado) .....                  | 47        |
| 3.5.4 Dependencias principales.....                      | 48        |
| 3.5.5 Métricas de control .....                          | 49        |
| 3.5.6 Calendario de ceremonias (adaptado).....           | 49        |
| 3.5.7 Gestión de buffers y contingencias .....           | 49        |
| <b>3.6 Plan de pruebas.....</b>                          | <b>49</b> |

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| 3.6.1 Alcance y objetivos .....                                   | 49        |
| 3.6.2 Estrategia y niveles.....                                   | 50        |
| 3.6.3 Entorno de pruebas .....                                    | 50        |
| 3.6.4 Diseño de casos de prueba funcional (PF) .....              | 50        |
| <b>3.6.5 Pruebas de integración / end-to-end (PI) .....</b>       | <b>51</b> |
| 3.6.6 Pruebas de aceptación (UAT).....                            | 51        |
| 3.6.7 Gestión de defectos .....                                   | 52        |
| 3.6.8 Criterios de salida por nivel.....                          | 52        |
| 3.6.9 Evidencias y trazabilidad .....                             | 52        |
| <b>Tabla modelo para registrar resultados .....</b>               | <b>53</b> |
| <b>3.7 Plan de riesgos.....</b>                                   | <b>53</b> |
| 3.7.1 Identificación de los riesgos.....                          | 53        |
| 3.7.2 Evaluación de los riesgos .....                             | 54        |
| <b>3.7.3 Plan de respuesta y contingencia.....</b>                | <b>54</b> |
| <b>3.7.4 Síntesis .....</b>                                       | <b>55</b> |
| <b>CAPÍTULO IV GESTIÓN DEL DESARROLLO DEL SISTEMA .....</b>       | <b>57</b> |
| 4.1 Sprint 1 – Login (RUT + PIN).....                             | 57        |
| 4.1.1 Planificación.....                                          | 57        |
| 4.1.2 Diseño de vistas .....                                      | 57        |
| 4.1.3 Riesgos.....                                                | 58        |
| 4.1.4 Evidencia de liberación del producto .....                  | 58        |
| 4.1.5 Evidencia del ciclo de desarrollo del Sprint 1 .....        | 59        |
| <b>4.1.6 Testing Sprint 1 .....</b>                               | <b>60</b> |
| 4.2 Sprint 2 – Gestión de productos y proveedores.....            | 60        |
| 4.2.1 Planificación.....                                          | 60        |
| 4.2.2 Diseño de vistas .....                                      | 60        |
| 4.2.3 Riesgos.....                                                | 61        |
| 4.2.4 Evidencia de liberación del producto .....                  | 62        |
| 4.2.5 Evidencia del ciclo de desarrollo del Sprint 2 .....        | 62        |
| <b>4.2.6 Testing Sprint 2 .....</b>                               | <b>62</b> |
| 4.3 Sprint 3 – Consumos históricos y control de stock mínimo..... | 62        |
| 4.3.1 Planificación.....                                          | 62        |
| 4.3.2 Diseño de vistas .....                                      | 63        |
| 4.3.3 Riesgos.....                                                | 63        |
| 4.3.4 Evidencia de liberación del producto .....                  | 64        |
| 4.3.5 Evidencia del ciclo de desarrollo del Sprint 3 .....        | 64        |
| <b>4.3.6 Testing Sprint 3 .....</b>                               | <b>64</b> |
| 4.4 Sprint 4 – Motor de sugerencias de pedidos (descartado) ..... | 65        |
| 4.4.1 Planificación.....                                          | 65        |
| 4.4.2 Avance y diseño preliminar.....                             | 65        |

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| 4.4.3 Riesgos identificados .....                                 | 65        |
| 4.4.4 Decisión de descarte.....                                   | 66        |
| 4.4.5 Evidencias de trabajo parcial.....                          | 66        |
| 4.4.6 Conclusión del Sprint 4.....                                | 66        |
| <b>4.5 Sprint 5 – Exportación de pedidos a Excel .....</b>        | <b>67</b> |
| 4.5.1 Planificación.....                                          | 67        |
| 4.5.2 Diseño de vistas .....                                      | 67        |
| 4.5.3 Riesgos.....                                                | 67        |
| 4.5.4 Evidencia de liberación del producto .....                  | 68        |
| 4.5.5 Evidencia del ciclo de desarrollo del Sprint 5 .....        | 68        |
| <b>4.5.6 Testing Sprint 5 .....</b>                               | <b>69</b> |
| 4.5.7 Conclusión del Sprint 5.....                                | 69        |
| <b>4.6 Sprint 6 – Alertas automáticas de stock crítico .....</b>  | <b>69</b> |
| 4.6.1 Planificación.....                                          | 69        |
| 4.6.2 Diseño de vistas .....                                      | 70        |
| 4.6.3 Riesgos.....                                                | 70        |
| 4.6.4 Evidencia de liberación del producto .....                  | 70        |
| 4.6.5 Evidencia del ciclo de desarrollo del Sprint 6 .....        | 70        |
| <b>4.6.6 Testing Sprint 6 .....</b>                               | <b>71</b> |
| 4.6.7 Conclusión del Sprint 6.....                                | 71        |
| <b>CAPÍTULO V ANÁLISIS DE RESULTADOS &amp; CONCLUSIONES .....</b> | <b>72</b> |
| 5.1 Resultados de los objetivos específicos SMART .....           | 72        |
| 5.2 Análisis post-mortem .....                                    | 73        |
| Fortalezas .....                                                  | 73        |
| Debilidades .....                                                 | 73        |
| Lecciones aprendidas .....                                        | 73        |
| 5.3 Conclusión .....                                              | 74        |
| 5.4 Trabajos futuros.....                                         | 75        |
| 5.5 Anexos.....                                                   | 75        |

## ÍNDICE DE FIGURAS

- **Figura 1.** Diagrama de Ishikawa (Fuente: Elaboración propia).
- **Figura 2.** Diagrama de Casos de Uso de ControlRepro (Fuente: Elaboración propia).
- **Figura 3.** Diagrama de Despliegue actual vs futuro de ControlRepro (Fuente: Elaboración propia).
- **Figura 4.** Diseño de alto nivel de ControlRepro (Fuente: Elaboración propia).
- **Figura 5.** Diagrama de clases de ControlRepro (Fuente: Elaboración propia).
- **Figura 6.** Diagrama de clases simplificado de ControlRepro (Fuente: Elaboración propia).

- **Figura 7.** Diagrama de secuencia – generación de pedido (Fuente: Elaboración propia).
- **Figura 8.** Diagrama de actividades – Flujo de generación de pedidos en ControlRepro (Fuente: Elaboración propia).
- **Figura 9.** Ciclo de Desarrollo Incremental aplicado a ControlRepro (Fuente: Elaboración propia).
- **Figura 10.** Esquema de Scrum adaptado a ControlRepro (Fuente: Elaboración propia).
- **Figura 11.** Cronograma Gantt de planificación del proyecto ControlRepro (Fuente: Elaboración propia).
- **Figura 12.** Matriz de Riesgos del proyecto ControlRepro (Fuente: Elaboración propia).
- **Figura 13.** Pantalla de Login de ControlRepro (Fuente: Captura del prototipo funcional).
- **Figura 14.** Inicio de sesión fallido en la aplicación de ControlRepro (Fuente: Captura del prototipo funcional).
- **Figura 15.** Formulario de productos en ControlRepro (Fuente: Captura del prototipo funcional).
- **Figura 16.** Formulario y listado de proveedores en ControlRepro (Fuente: Captura del prototipo funcional).
- **Figura 17.** Productos con su proveedor asociado y lead time (Fuente: Captura del prototipo funcional).
- **Figura 18.** Formulario de registro de consumo en ControlRepro (Fuente: Captura del prototipo funcional).
- **Figura 19.** Evidencia exportación Excel (Fuente: Captura del prototipo funcional).

## ÍNDICE DE TABLAS

- **Tabla 1.** Matriz de trazabilidad de objetivos funcionales.
- **Tabla 2.** Matriz de trazabilidad de objetivos no funcionales.
- **Tabla 3.** Resumen de factibilidades de ControlRepro.
- **Tabla 4.** Modelo de plan de pruebas – resultados de casos.
- **Tabla 5.** Identificación de riesgos.
- **Tabla 6.** Evaluación de riesgos (probabilidad, impacto, nivel).
- **Tabla 7.** Plan de respuesta y contingencia de riesgos.
- **Tabla 8.** Resultados de objetivos específicos SMART.
- **Tabla 9.** Casos de prueba funcionales (PF).
- **Tabla 10.** Casos de prueba de integración (PI).
- **Tabla 11.** Casos de prueba de aceptación (UAT).

## **Resumen**

El proyecto ControlRepro tiene como propósito desarrollar una aplicación multiplataforma orientada a pequeñas y medianas empresas (PYMES), cuyo objetivo principal es optimizar la reposición de productos a través de la generación automática de pedidos a proveedores. La solución surge frente a la problemática recurrente de este tipo de organizaciones: la ausencia de herramientas tecnológicas accesibles para la gestión de inventarios, lo que provoca errores humanos, quiebres de stock y pérdidas económicas.

Para abordar este desafío, se diseñó un sistema autónomo y simple de utilizar, capaz de operar sin conexión a internet. La aplicación permite registrar productos y proveedores, mantener un historial de consumos, establecer puntos mínimos de stock y calcular pedidos sugeridos considerando además los tiempos de entrega de cada proveedor. Con ello, se busca garantizar continuidad operativa y apoyar la toma de decisiones mediante información confiable y trazable.

El desarrollo del proyecto se realizó siguiendo un enfoque ágil e incremental, estructurado en sprints semanales que permitieron validar progresivamente cada módulo. Se aplicaron técnicas de modelado UML para representar casos de uso, procesos y entidades, y se construyó un prototipo funcional bajo un diseño modular que facilita su escalabilidad futura.

En definitiva, ControlRepro constituye una propuesta tecnológica pertinente al contexto de las PYMES chilenas, al ofrecer una herramienta que sistematiza el controinventarios, reduce errores en la reposición de productos y contribuye a mejorar la eficiencia y competitividad de estas organizaciones.

# CAPÍTULO I INTRODUCCIÓN

## 1. Introducción

El desarrollo de sistemas tecnológicos orientados a resolver problemáticas operativas en las pequeñas y medianas empresas (PYMES) se ha convertido en un factor clave para su sostenibilidad y crecimiento. En Chile, una gran parte de estas organizaciones aún carece de herramientas digitales que les permitan gestionar de forma eficiente aspectos críticos de su funcionamiento, como lo es el control de stock y la reposición de productos. Esta deficiencia se traduce en quiebres de stock, sobreabastecimientos, decisiones reactivas y, en última instancia, pérdidas económicas y pérdida de competitividad frente a empresas más tecnificadas.

De acuerdo con SERCOTEC (2023), más del 60% de las PYMES chilenas carecen de procesos de digitalización básicos, especialmente en áreas relacionadas con la gestión de inventario. CORFO, a través de su programa Digitaliza tu Pyme, también ha advertido que la falta de herramientas accesibles para la gestión logística constituye uno de los principales obstáculos para mejorar la productividad de este sector. Asimismo, SUBTEL ha señalado que, a pesar del aumento en la conectividad, muchas pequeñas empresas siguen dependiendo de procesos manuales debido a la ausencia de soluciones offline que se adapten a contextos con baja cobertura. Estos antecedentes refuerzan la pertinencia de una propuesta como ControlRepro en el ecosistema nacional.

En este contexto, el presente proyecto propone el desarrollo de ControlRepro, una aplicación multiplataforma que busca automatizar la generación de pedidos a proveedores utilizando como base el historial de consumo, el stock mínimo definido y los tiempos de entrega (lead time). Su enfoque está especialmente diseñado para adaptarse a las condiciones reales de las PYMES: sin requerimientos de conexión a internet, con una interfaz intuitiva, y sin necesidad de conocimientos técnicos avanzados por parte de los usuarios. Esta propuesta representa una respuesta directa a las conclusiones observadas durante la fase diagnóstica del curso, donde se identificó una alta disposición de las PYMES a adoptar soluciones tecnológicas siempre que estas sean simples, accesibles y funcionales.



## 1.1 Contexto

Las PYMES constituyen el 98% del tejido empresarial en Chile y generan una parte significativa del empleo nacional. Sin embargo, la mayoría aún opera con procesos manuales para la gestión de inventario y reposición de productos, lo que provoca problemas recurrentes:

Falta de planificación en reposición: se generan pedidos tardíos por ausencia de control sistemático.

Errores humanos: registros incompletos o duplicados al llevar el control en planillas Excel.

Rupturas de stock: productos agotados que afectan directamente la continuidad operativa y la atención al cliente.

Ausencia de trazabilidad: no existe un registro histórico confiable de consumos ni de pedidos asociados a responsables.

En este escenario, el encargado de bodega o el administrador suele depender de su criterio personal para decidir cuándo y cuánto reponer, sin un sistema que lo respalde en la toma de decisiones. Al no existir un flujo estandarizado, los registros se pierden o quedan incompletos, lo que genera retrasos en la entrega a clientes y afecta la sostenibilidad del negocio.

El proyecto ControlRepro nace como una respuesta a estas dificultades, ofreciendo una herramienta que permita sistematizar el proceso de reposición, reducir errores humanos y entregar información confiable para la gestión de compras.

## 1.2 Definición del problema

Las pequeñas y medianas empresas (PYMES) en Chile enfrentan dificultades recurrentes en la gestión de sus inventarios debido a la falta de herramientas tecnológicas accesibles. La mayoría de ellas aún depende de procesos manuales o registros en planillas Excel, lo que genera información incompleta, errores en la reposición, pedidos tardíos y pérdidas económicas.

Uno de los principales riesgos es la **falta de trazabilidad**: los consumos de productos y los pedidos a proveedores no quedan asociados de manera confiable a un responsable, lo que impide medir tiempos de respuesta y dificulta identificar las causas de los quiebres de stock. Asimismo, la ausencia de controles automáticos incrementa la probabilidad de errores humanos y de sobrecargar el trabajo de los encargados de bodega.

Para visualizar las causas raíz de esta problemática, se elaboró un **Diagrama de Ishikawa**, en el cual se identifican los factores más relevantes que explican las fallas en los procesos de reposición. Entre ellos destacan:

- **Métodos y procesos inadecuados**, sin estándares claros.
- **Errores humanos** derivados de la gestión manual.
- **Problemas con proveedores**, asociados a tiempos de entrega.
- **Uso de tecnología insuficiente**, limitada a planillas no integradas.
- **Factores externos**, como variaciones de demanda o imprevistos logísticos.

Este análisis confirma la necesidad de una solución tecnológica como **ControlRepro**, que permita sistematizar la reposición de productos, reducir errores humanos y garantizar información confiable para la toma de decisiones.

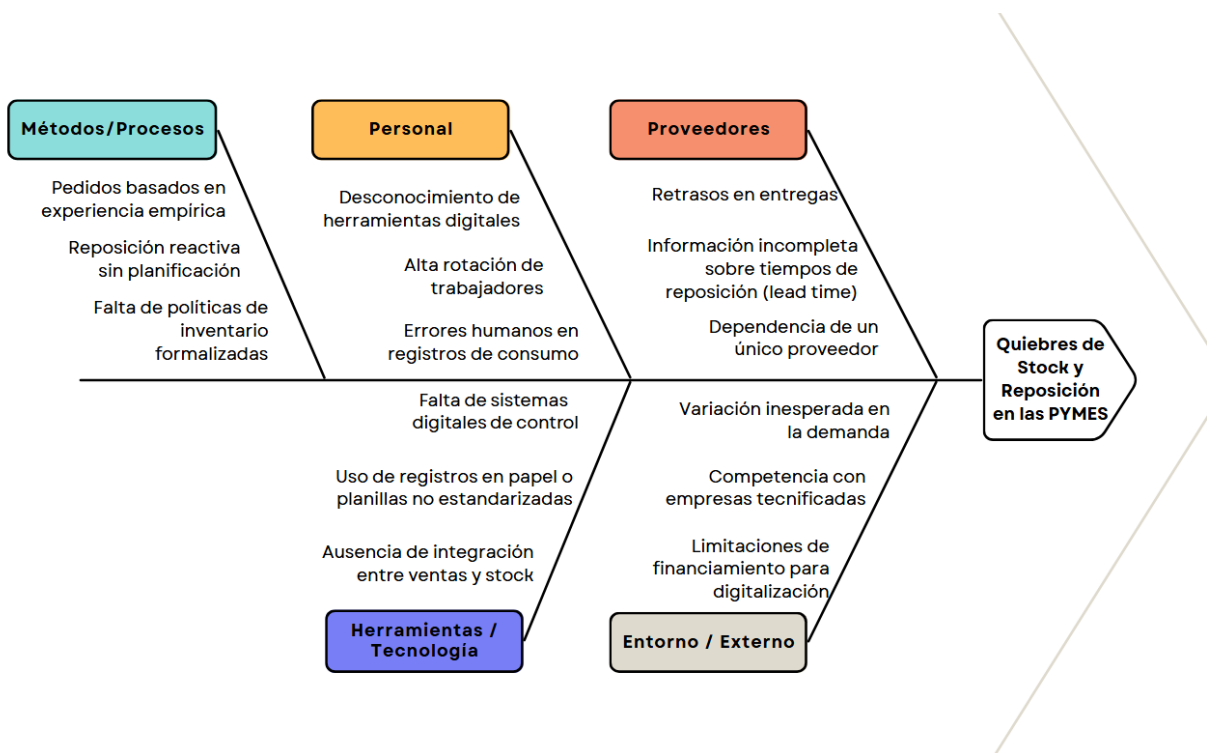


Figura 1. Diagrama de Ishikawa – Fuente: Elaboración propia.

## 1.4 OBJETIVOS

### 1.4.1 Objetivo General

Desarrollar una aplicación multiplataforma denominada **ControlRepro**, orientada a pequeñas y medianas empresas (PYMES) de Chile, que permita optimizar el proceso de reposición de productos mediante la digitalización del registro de consumos, la gestión de proveedores y el uso de parámetros como stock mínimo y tiempos de entrega (*lead time*). Con esta solución tecnológica se busca reducir las pérdidas económicas ocasionadas por quiebres de stock, minimizar los errores derivados de la gestión manual en planillas, y entregar información confiable y trazable que respalde la toma de decisiones de los administradores y encargados de inventario.

### 1.4.2 Objetivos Específicos

1. **Implementar un sistema de autenticación seguro y trazable**, basado en el ingreso de RUT y PIN, que permita garantizar que cada operación registrada quede asociada a un trabajador identificado. Este objetivo busca no solo mejorar la seguridad del sistema, sino también asegurar la responsabilidad y el control de cada acción realizada dentro de la aplicación.
2. **Diseñar y desarrollar un módulo integral de gestión de productos y proveedores**, que permita registrar de manera estructurada las características de cada producto, su proveedor asociado y parámetros clave como el *lead time*. De esta manera, se busca asegurar que la reposición esté respaldada por información confiable, evitando duplicidades, errores o datos incompletos.
3. **Construir un registro digital de consumos históricos**, en el que se almacenen las salidas de productos con detalle de cantidad, fecha y trabajador responsable. Este registro busca reemplazar los sistemas manuales en planillas, permitiendo contar con un historial ordenado y auditable, fundamental para el cálculo de reposiciones y la identificación de patrones de consumo.
4. **Implementar un motor de generación automática de pedidos sugeridos**, que utilice como base los datos de consumo histórico, el stock mínimo definido y el *lead time* de cada proveedor. Con esta funcionalidad, se busca anticipar necesidades de compra, reducir quiebres de stock y agilizar la preparación de pedidos, aportando a la continuidad operativa de las empresas.
5. **Incorporar un sistema de exportación de pedidos en formatos Excel y PDF**, que entregue documentos listos para ser enviados a proveedores. Estos pedidos incluirán información relevante como productos, cantidades solicitadas, precios y responsable de la operación, con el fin de mejorar la comunicación con los proveedores y respaldar las decisiones administrativas.

6. **Desarrollar un sistema de alertas automáticas y notificaciones internas**, que informe al usuario cuando un producto se encuentre en nivel crítico de stock o cuando se presenten anomalías en los registros. Este objetivo apunta a mejorar la capacidad de reacción de la empresa, evitando pérdidas por quiebres y mejorando la planificación de las compras.
7. **Diseñar la arquitectura del sistema bajo el patrón Modelo-Vista-Controlador (MVC) y con base de datos local en SQLite**, garantizando así la autonomía del sistema, su escalabilidad futura y la facilidad de mantenimiento. Este objetivo busca sentar las bases de un software robusto, flexible y preparado para evolucionar hacia esquemas más complejos (como integración en la nube) sin perder su simplicidad inicial.

### 1.4.3 Matriz de Trazabilidad de Objetivos

#### Matriz de Trazabilidad – Objetivos Funcionales

| ID Req | Tipo      | Descripción detallada                                                                                                                                                     | Sprint / Iteración | Artefactos UML asociados                                                                                                | Componentes de diseño / Evidencias                                                                                                                                |
|--------|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|-------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RF-01  | Funcional | Validar RUT chileno en formato <b>12.345.678-9</b> y PIN de acceso para garantizar seguridad en la autenticación. Incluye validación en tiempo real y registro de sesión. | Sprint 1 / Hito 2  | Caso de uso: <i>Iniciar sesión</i> ; Diagrama de secuencia: <i>Autenticación</i>                                        | Clase <b>Usuario</b> , Controlador <b>Auth</b> ; BD tabla <i>trabajadores</i> (rut, pin). Evidencia: mockup login, manual de usuario, logs de sesión.             |
| RF-02  | Funcional | Solo permitir registrar productos ya existentes en la base de datos, evitando creación duplicada o errónea. Campo de búsqueda bloqueado tras selección.                   | Sprint 2 / Hito 3  | Caso de uso: <i>Registrar producto</i> ; Diagrama de clases: <i>Producto</i> ; Secuencia: <i>Validación de producto</i> | Clase <b>Producto</b> , DAO <i>Producto</i> ; Tabla <i>productos</i> (id, nombre, stock, proveedor). Evidencia: mockup CRUD productos, control de autocompletado. |

| ID Req | Tipo      | Descripción detallada                                                                                                                                        | Sprint / Iteración | Artefactos UML asociados                                                                                                                    | Componentes de diseño / Evidencias                                                                                                                                                           |
|--------|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RF-03  | Funcional | Registrar consumos históricos de productos, vinculando fecha, cantidad y trabajador responsable. Validar stock mínimo y evitar negativos.                    | Sprint 3 / Hito 4  | Caso de uso: <i>Registrar consumo</i> ; Secuencia: <i>Flujo de entrada/salida</i> ; Diagrama de clases: <i>Consumo</i>                      | Clases <b>Producto</b> , <b>Consumo</b> , <b>Usuario</b> ; Tabla <i>consumos</i> (id, fecha, cantidad, rut_trabajador). Evidencia: log histórico, mockup registro consumo, manual operativo. |
| RF-04  | Funcional | Emitir alertas automáticas cuando el stock actual $\leq$ stock mínimo, o cuando proveedor presenta retrasos. Se muestran en Dashboard y tabla de alertas.    | Sprint 3 / Hito 7  | Caso de uso: <i>Visualizar alertas</i> ; Secuencia: <i>Registrar consumo</i> (con extensión alerta)                                         | Clase <b>Producto</b> , UI Alertas; BD: tabla <i>alertas</i> . Evidencia: mockup Dashboard con badge "Stock Crítico".                                                                        |
| RF-05  | Funcional | Generar pedidos sugeridos en base a parámetros: consumo histórico, stock mínimo y lead time del proveedor. Calcular cantidad óptima $\geq 90\%$ precisión.   | Sprint 4 / Hito 5  | Caso de uso: <i>Generar pedido sugerido</i> ; Secuencia: <i>Generación de pedido</i> ; Diagrama de clases: <i>Pedido</i> , <i>Proveedor</i> | Clases <b>Pedido</b> , <b>PedidoDetalle</b> , <b>ProductoProveedor</b> ; BD: tabla <i>pedidos</i> . Evidencia: mockup generación de pedidos, pruebas de precisión.                           |
| RF-06  | Funcional | Exportar pedidos confirmados a <b>Excel/PDF</b> , incluyendo: productos, cantidades, precios y trabajador responsable. Archivo debe guardarse en escritorio. | Sprint 4 / Hito 6  | Caso de uso: <i>Exportar pedido</i> ; Secuencia: <i>Exportación</i> ; Mockup pantalla exportación                                           | Clase <b>ServicioExportación</b> , FileSystem local, librerías <b>xlsx/pdfkit</b> . Evidencia: archivos generados con nombre (RUT + fecha), manual de exportación.                           |

Nota: la trazabilidad por trabajador se planificó, pero en el MVP solo se implementó registro de consumos; la vinculación al usuario se documenta como pendiente/futura

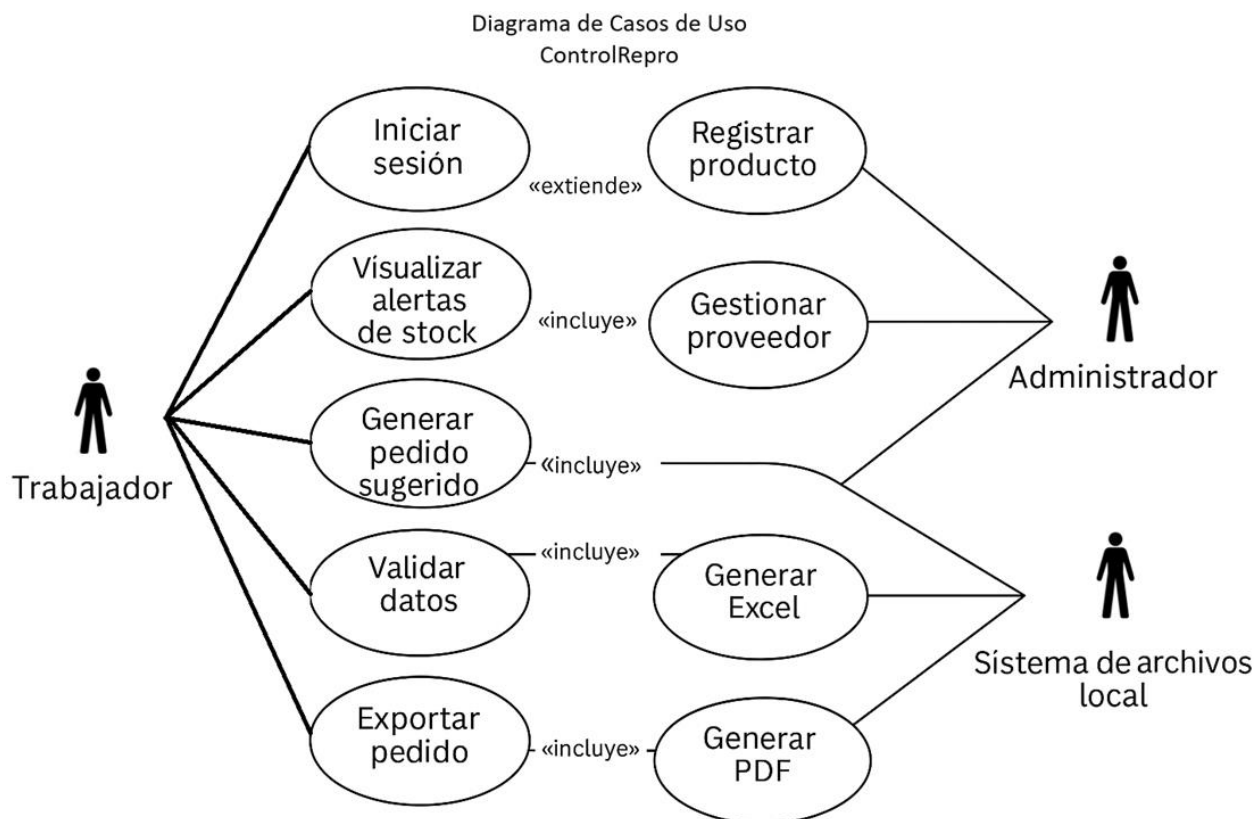


Figura 2. Diagrama de Casos de Uso de ControlRepro (Fuente: Elaboración propia).

### Matriz de Trazabilidad de Objetivos No Funcionales

| ID Req | Tipo         | Descripción detallada                                                                                                           | Sprint Iteración                      | Artefactos UML asociados                                               | Componentes de diseño / Evidencias                                                                                     |
|--------|--------------|---------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| RNF-01 | No funcional | La aplicación debe ser <b>100% operativa sin conexión a internet</b> , garantizando autonomía para PYMES con baja conectividad. | Todos Sprints<br>Arquitectura inicial | Diagrama de despliegue (3 capas offline: presentación, lógica, datos). | Arquitectura local: <b>Electron + Node.js + SQLite</b> . Evidencia: manual de instalación, pruebas en entorno sin red. |
| RNF-02 | No funcional | El nombre y RUT del trabajador deben quedar registrados en cada operación (consumos, pedidos, exportaciones).                   | Sprints 1-4                           | Diagrama de clases: <i>Usuario</i> , <i>Consumo</i> ,                  | BD: campos <i>rut_trabajador</i> en consumos y pedidos. Evidencia: logs locales, archivo                               |

| ID Req | Tipo        | Descripción detallada                                                                                             | Sprint Iteración     | Artefactos UML asociados                                          | Componentes de diseño / Evidencias                                                                                                                 |
|--------|-------------|-------------------------------------------------------------------------------------------------------------------|----------------------|-------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
|        |             |                                                                                                                   |                      | <i>Pedido</i> con usuariold.                                      | exportado con RUT en nombre.                                                                                                                       |
| RNF-03 | No funciona | La interfaz debe ser <b>intuitiva y usable</b> , con validaciones claras y mensajes de error explícitos.          | Todos / Mockups      | Diagramas de casos de uso (interacciones de usuario); Mockups UI. | UI construida en <b>Vue/Electron</b> , estilos <b>Tailwind CSS</b> . Evidencia: mockups login, dashboard, pruebas UAT (≤2 minutos generar pedido). |
| RNF-04 | No funciona | Los archivos exportados deben guardarse en el escritorio del usuario, con estructura "pedido_RUT_fecha.xlsx/pdf". | Sprint 4 / Hito 6    | Caso de uso: <i>Exportar pedido</i> ; Secuencia de exportación.   | Lógica en <b>xlsx/pdfkit</b> , FileSystem local. Evidencia: archivos de prueba generados, manual operativo.                                        |
| RNF-05 | No funciona | El sistema debe ser <b>escalable</b> a futuro hacia arquitectura cliente-servidor (PostgreSQL/MySQL + Cloud).     | Evolución proyectada | Diagrama de despliegue futuro (cliente-servidor + cloud).         | Infraestructura: <b>PostgreSQL/MySQL</b> , <b>AWS/Azure</b> . Evidencia: proyección de arquitectura, documentación técnica.                        |

Nota: inicialmente se consideró incluir un **motor de sugerencias automáticas de pedidos** (RF-05) y la exportación en formato PDF, sin embargo, tras la evaluación técnica y de alcance, ambas funcionalidades fueron **descartadas del MVP**. Se mantuvo únicamente la exportación a Excel, y el motor de sugerencias se proyecta como mejora futura

#### 1.4.4 Calidad de los Objetivos

**Propósito de la sección.** Evaluar críticamente la calidad del **objetivo general** y los **objetivos específicos** definidos para ControlRepro, verificando su alineamiento con el problema diagnosticado, su carácter **SMART**, su **consistencia interna**, **viabilidad** y **trazabilidad** hacia diseño, planes de prueba y entregables del MVP. Como base se emplean los objetivos establecidos en las Partes 1–3 del informe y su planificación por sprints.

## A. Criterios metodológicos utilizados

### 1. SMART

- **Específico:** describe con claridad “qué” se logrará.
  - **Medible:** define métricas o evidencias objetivas para verificar cumplimiento.
  - **Alcanzable:** posible con recursos/tecnologías declaradas.
  - **Relevante:** responde al problema de reposición en PYMES.
  - **Temporal:** acotado a la duración del proyecto (12 semanas / sprints).
2. **Consistencia y no ambigüedad:** objetivos sin contradicciones ni solapamientos innecesarios.
  3. **Viabilidad (técnica, operacional):** factible con **Electron + Node.js + SQLite** en modo offline (MVP).
  4. **Trazabilidad y verificabilidad:** cada objetivo se vincula a módulos, artefactos UML y casos de prueba funcional/UAT.

## B. Evaluación del Objetivo General

**Desarrollar una aplicación multiplataforma** para automatizar la reposición mediante **consumo histórico, stock mínimo y lead time**, reduciendo quiebres de stock en PYMES, dentro de **12 semanas**.

- **S / M / T:** Es claro, cuantificable en tiempo (12 semanas) y validable por la presencia de los módulos mínimos del MVP (login, productos/proveedores, consumos, sugerencias/lead time, exportación, alertas).
- **A:** Alcanzable con la arquitectura local declarada y requerimientos de hardware mínimos.
- **R:** Directamente relevante al problema diagnosticado de reposiciones no sistematizadas en PYMES.
- **Trazabilidad:** el objetivo general se descompone en sprints/entregables con hitos y pruebas funcionales, lo que permite verificación incremental.

**Conclusión:** Objetivo general **SMART, consistente y verificable**, con soporte técnico y operativo suficiente para su logro en el plazo establecido.

## C. Evaluación de Objetivos Específicos

Para cada objetivo se indica: **SMART, viabilidad y evidencias de verificación**.

### 1. Login seguro (RUT + PIN) en ≤3 semanas.



- **S/M/T:** definido en alcance y plazo; verificable por **PF-01 Login** (acceso con credenciales válidas / error con inválidas).
- **A:** viable en arquitectura local con validación de formato 12.345.678-9 y PIN; evidencia en manual de usuario.
- **R:** requisito transversal para trazabilidad de operaciones (responsable por RUT).
- 2. **Gestión de productos y proveedores con lead time ( $\leq 4$  semanas).**
  - **S/M/T:** tareas y artefactos definidos (clase Producto, DAO, casos de uso).
  - **A:** base de datos relacional y UI CRUD declaradas; pruebas PF-02/PF-03.
  - **R:** imprescindible para calcular reposiciones por proveedor (lead time).
- 3. **Registro de consumos históricos ( $\geq 10$  registros simulados).**
  - **S/M:** entidad Consumo con campos y restricciones (fecha ISO-8601, cantidad  $> 0$ , rut\_trabajador FK).
  - **A:** flujo operativo definido con pre/postcondiciones.
  - **Verificación:** PF-04 “Registro de Consumo” y PI-01 “Consumo  $\rightarrow$  afecta sugerencias”.
- 4. **Motor de sugerencias / cálculo con stock mínimo, consumo histórico y lead time ( $\leq 6-8$  semanas).**
  - **S/M:** caso de uso, secuencia y clases de Pedido/Proveedor definidos; métrica de precisión en escenarios de prueba.
  - **A:** calculable en backend local; se reconoce que el proyecto puede simplificar priorizando **lead time** (riesgo gestionado).
  - **Verificación:** PF-05 “Pedido Sugerido”, PI-02 asociación Producto–Proveedor y UAT flujo completo.
- 5. **Exportación de pedidos a Excel/PDF ( $\leq 8$  semanas).**
  - **S/M:** contenido mínimo del archivo y localización en escritorio definidos; casos PF-06.
  - **A:** viable con **xlsx/pdftkit** y permisos locales; evidencia en manual de instalación/monitorización.
- 6. **Alertas automáticas por stock crítico / retrasos.** (planificado hacia semana 9)
  - **S/M:** control de verificación de stock vs mínimo y evidencia en UI “Stock Crítico”.
  - **A:** UI/Badge en Dashboard + tabla de alertas; evidencia en manual de usuario.

**Conclusión por objetivos específicos:** todos cumplen **SMART**; su factibilidad técnica se respalda en la arquitectura y modelo de datos; y su verificación está cubierta por **pruebas funcionales, de integración y UAT** incluidas en la Parte 3.

## D. Consistencia interna y cobertura del problema

Los objetivos no se contradicen y siguen una **dependencia lógica**: autenticación  $\rightarrow$  maestros (productos/proveedores)  $\rightarrow$  captura de consumos  $\rightarrow$  cálculo de reposiciones  $\rightarrow$  exportación  $\rightarrow$  alertas; exactamente el orden de construcción planificado por sprints. Esto asegura **trazabilidad y control de riesgos** durante la entrega incremental.

## E. Viabilidad y riesgos controlados

- **Técnica/Operacional:** implementación offline con Electron/Node/SQLite, requisitos mínimos y procedimientos de instalación/uso documentados.
- **Riesgo reconocido:** complejidad del motor de sugerencias; se contempla mitigación priorizando la heurística por **lead time** y validación empírica en PF-05/UAT.

## F. Verificabilidad (métricas y evidencias)

Cada objetivo cuenta con **artefactos UML**, **componentes de diseño**, **casos de prueba** y **evidencias** (capturas/archivos exportados, logs) para su verificación objetiva, tal como se consolidó en la **matriz de trazabilidad** y en el **plan de pruebas** del proyecto.

**Síntesis.** Los objetivos del proyecto ControlRepro poseen **alta calidad**: son **SMART**, consistentes, pertinentes al problema de negocio, **viabiles** con la arquitectura y herramientas definidas y **plenamente trazables** a diseño, pruebas y entregables del MVP. Esto otorga una base sólida para avanzar al resto del documento y a la implementación.

## 1.5 Alcance y Limitaciones

### 1.5.1 Alcance

El alcance del proyecto **ControlRepro** está definido bajo los criterios de un **Producto Mínimo Viable (MVP)** a implementar en un periodo de 12 semanas. Su objetivo es entregar a las pequeñas y medianas empresas (PYMES) una herramienta funcional, simple y autónoma para la gestión de reposición de inventario.

En esta primera etapa, el sistema contempla:

- **Acceso seguro mediante login local** con autenticación por **RUT y PIN**, validando la identidad del trabajador y registrando la trazabilidad de cada acción.
- **Gestión de productos y proveedores**, incluyendo la definición de stock mínimo, precios por caja y **lead time** de entrega, lo que permite establecer parámetros claros para la reposición.
- **Registro de consumos históricos**, con identificación del trabajador responsable y almacenamiento local de al menos 10 consumos simulados para validar trazabilidad.
- **Generación de pedidos sugeridos**, basada en los parámetros definidos (stock mínimo, consumo histórico y lead time), orientada a disminuir quiebres de stock y optimizar la reposición.
- **Exportación de pedidos a Excel y PDF**, con estructura estandarizada que incluye productos, cantidades, proveedor y responsable de la acción.

- **Sistema de alertas automáticas**, que notifica al usuario cuando el stock actual es igual o inferior al stock mínimo establecido, facilitando una toma de decisiones proactiva.
- **Arquitectura local (offline)**, ejecutándose como una aplicación de escritorio autónoma en Windows/Linux, sin necesidad de conexión a internet ni infraestructura externa.
- **Soporte documental**, compuesto por manual de usuario, manual de instalación, procedimientos operativos y guías básicas de monitorización.

De esta forma, el alcance del MVP asegura que el sistema cumpla con las necesidades esenciales detectadas en las PYMES: simplicidad de uso, autonomía de operación y trazabilidad de las acciones críticas en la reposición de productos.

### 1.5.2 Limitaciones

El proyecto reconoce explícitamente un conjunto de limitaciones propias de su condición de MVP, que serán consideradas como oportunidades de mejora en futuras fases de evolución:

- **Conectividad y escalabilidad limitada**: el sistema funciona en modo local sin conexión a internet. No contempla integración con servicios en la nube, bases de datos centralizadas ni sincronización entre múltiples equipos.
- **Gestión de usuarios básica**: el control de acceso se limita a trabajadores identificados por RUT y PIN. No existen perfiles jerárquicos avanzados, roles diferenciados ni autenticación multifactor.
- **Motor de sugerencias simplificado**: aunque se utiliza el consumo histórico, stock mínimo y lead time, el cálculo no incorpora algoritmos predictivos avanzados (machine learning, análisis de estacionalidad, series temporales, etc.).
- **Monitoreo y métricas**: el registro de eventos se limita a logs locales. No se dispone de tableros de monitoreo, auditorías centralizadas ni métricas en tiempo real.
- **Capacidad de integración reducida**: el sistema no se integra aún con software de ventas, facturación o contabilidad externos, ni contempla conexión con proveedores mediante APIs.
- **Escalabilidad técnica limitada**: la base de datos SQLite satisface las necesidades de un entorno PYME reducido, pero no está diseñada para cargas masivas o múltiples instancias concurrentes.
- **Pruebas en entorno controlado**: las validaciones se realizan con registros simulados y un conjunto limitado de escenarios, por lo que aún no se dispone de evidencia de uso en ambientes empresariales de gran escala.

#### Síntesis:

El alcance garantiza una herramienta **simple, funcional y validada** para PYMES en un escenario controlado y local, mientras que las limitaciones establecen de manera clara los aspectos que requieren futuras iteraciones para transformar ControlRepro en una solución escalable, conectada y de uso empresarial avanzado.

# CAPÍTULO II FUNDAMENTACION DEL TEMA

## 2. Fundamentación del tema

El desarrollo de soluciones tecnológicas orientadas a la gestión operativa de las pequeñas y medianas empresas (PYMES) constituye un eje central en los procesos de modernización productiva en Chile. Diversos estudios de organismos estatales y académicos han identificado que la ausencia de digitalización en procesos críticos, como la gestión de inventario y la reposición de productos, genera pérdidas económicas significativas, disminuye la competitividad y limita la capacidad de respuesta frente a un mercado cada vez más exigente y dinámico.

En este contexto, el proyecto **ControlRepro** surge como respuesta a la necesidad de contar con un sistema simple, accesible y adaptado a las condiciones reales de las PYMES, donde la mayoría de los procesos aún se realizan en planillas manuales o mediante registros informales. La dependencia de estas prácticas no solo aumenta la probabilidad de errores humanos, sino que también impide establecer mecanismos de trazabilidad, control y planificación de compras, elementos clave para mantener la continuidad operacional.

La fundamentación de este proyecto se sostiene en tres ejes principales:

- **Eje operativo:** la gestión manual de la reposición conlleva quiebres de stock, sobreabastecimientos y decisiones reactivas que afectan directamente los costos y el nivel de servicio al cliente. ControlRepro aborda este problema mediante la digitalización de registros de consumo y la automatización de pedidos sugeridos.
- **Eje tecnológico:** la aplicación se diseña bajo una arquitectura local (Electron, Node.js, SQLite), lo que permite operar sin conexión a internet y con recursos mínimos de hardware, alineándose con las limitaciones típicas de las PYMES que no disponen de infraestructura tecnológica avanzada.
- **Eje estratégico:** al proveer información confiable y trazable, ControlRepro fortalece la capacidad de los administradores para tomar decisiones informadas, mejorar la planificación de compras y reducir pérdidas económicas derivadas de fallas en el control de inventario.

La propuesta no busca competir con plataformas ERP de gran escala, las cuales suelen ser costosas y de difícil implementación en este segmento, sino entregar una herramienta ajustada a la realidad de las PYMES, que pueda ser utilizada sin conocimientos técnicos especializados y con la posibilidad de escalar en fases posteriores hacia arquitecturas más complejas (cliente-servidor, nube, integración con sistemas externos).

En síntesis, la fundamentación del tema establece que la implementación de **ControlRepro** no solo responde a una problemática concreta detectada en las PYMES chilenas, sino que también

representa una alternativa viable, técnicamente factible y estratégicamente pertinente. Este capítulo desarrolla en detalle la **solución propuesta**, analiza **soluciones de mercado comparables**, presenta el **diseño de alto nivel**, evalúa las **factibilidades** del proyecto y finalmente aplica el **modelo de vistas 4+1** como marco conceptual para validar la arquitectura planteada.

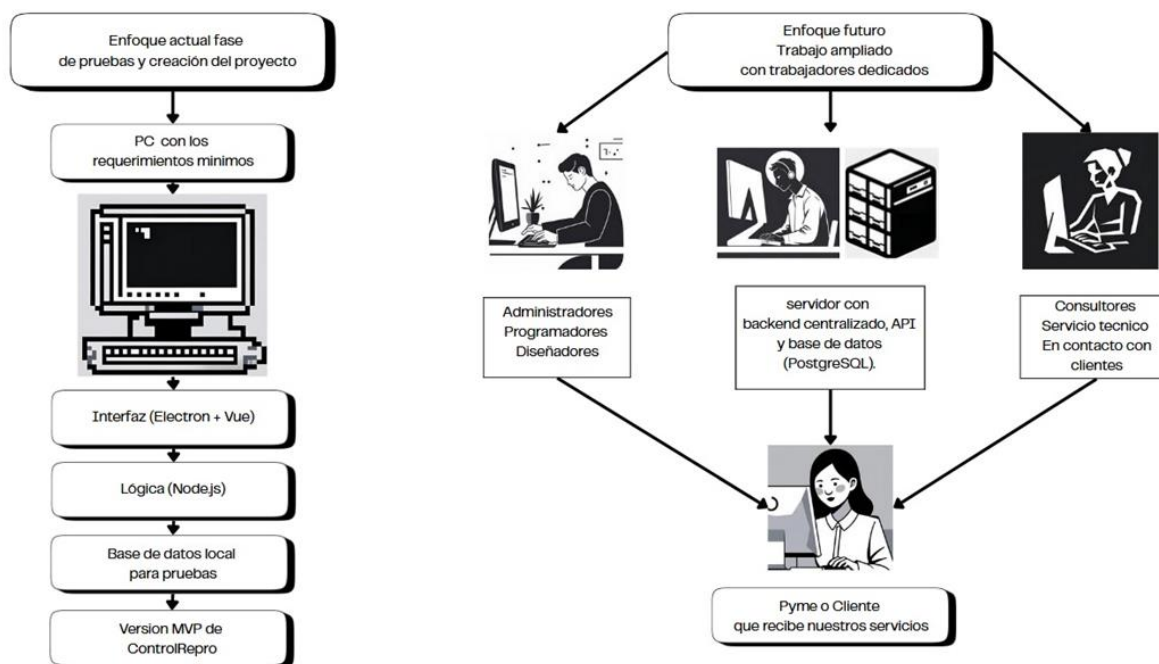


Figura 3. Diagrama de Despliegue actual vs futuro de ControlRepro (Fuente: Elaboración propia).

## 2.1 Solución propuesta

La solución propuesta consiste en el desarrollo de **ControlRepro**, una aplicación multiplataforma orientada a pequeñas y medianas empresas (PYMES) de Chile, cuyo objetivo principal es **optimizar la reposición de productos** a través de la digitalización de consumos históricos, la gestión de parámetros de inventario (stock mínimo) y la incorporación de tiempos de entrega (lead time) de proveedores.

La propuesta se fundamenta en un **modelo incremental de implementación** que permita alcanzar un **Producto Mínimo Viable (MVP)** en 12 semanas, con entregas progresivas de módulos funcionales. La aplicación se ejecuta de forma **local y autónoma**, sin necesidad de conexión a internet, lo que asegura su pertinencia para empresas que operan en entornos con baja conectividad o recursos tecnológicos limitados.

Componentes principales de la solución

### 1. Módulo de Login Seguro

- Autenticación mediante **RUT y PIN**, validando la identidad de cada trabajador.

- Registro de trazabilidad para asociar todas las operaciones a un responsable.
- Prevención de accesos no autorizados y control básico de sesiones activas.
- 2. **Gestión de Productos y Proveedores**
  - Administración de productos con parámetros clave: categoría, stock actual, stock mínimo y precio por caja.
  - Asociación de proveedores a cada producto, incluyendo tiempos de entrega (lead time).
  - Control estructurado de datos maestros que sirven de base para la generación de pedidos.
- 3. **Registro de Consumos Históricos**
  - Ingreso manual de consumos vinculados a un trabajador autenticado.
  - Almacenamiento en una base de datos local (SQLite) que garantiza integridad y trazabilidad.
  - Generación de un historial que permite proyectar patrones de consumo y detectar anomalías.
- 4. **Motor de Sugerencias de Reposición**
  - Generación automática de pedidos sugeridos en función de:
    - Stock mínimo definido.
    - Consumo histórico registrado.
    - Lead time del proveedor asociado.
  - Cálculo simplificado, orientado a garantizar continuidad operativa y evitar quiebres de stock.
- 5. **Exportación de Pedidos a Excel y PDF**
  - Creación de documentos listos para enviar a proveedores.
  - Inclusión de datos clave: productos solicitados, cantidades, precios y responsable de la operación.
  - Generación automática de archivos en el escritorio del usuario, con nomenclatura estándar (pedido\_RUT\_fecha).
- 6. **Sistema de Alertas Automáticas**
  - Notificación inmediata cuando el stock actual se encuentra en un nivel crítico.
  - Alertas visuales dentro del dashboard principal, asociadas a productos con riesgo de ruptura de stock.
  - Soporte para mejorar la reacción de la empresa ante imprevistos en el abastecimiento.

#### Enfoque tecnológico

- **Arquitectura:** Modelo Vista–Controlador (MVC), separando la lógica de negocio, la interfaz de usuario y la persistencia de datos.
- **Plataforma de desarrollo:** Node.js + Electron para el backend local, y React/Vue para la interfaz gráfica.
- **Base de datos:** SQLite, elegida por su simplicidad, autonomía y capacidad de funcionar sin conexión a internet.

- **Estilos visuales:** Tailwind CSS, lo que asegura una interfaz limpia, moderna y responsiva.
- **Exportación de documentos:** librerías xlsx y pdfkit.

Pertinencia y valor agregado

La propuesta de **ControlRepro** aporta valor a las PYMES mediante:

- **Accesibilidad:** aplicación local que no requiere infraestructura externa.
- **Simplicidad de uso:** interfaz intuitiva y pensada para trabajadores sin conocimientos técnicos avanzados.
- **Reducción de errores humanos:** reemplazo de procesos manuales en planillas por registros estructurados y validados.
- **Información trazable:** registro de cada operación con identificación de responsable.
- **Adaptabilidad:** posibilidad de evolucionar a futuro hacia una arquitectura cliente-servidor con integración en la nube.

En suma, **ControlRepro** se plantea como una solución pragmática y viable, diseñada para responder a las problemáticas de reposición en PYMES chilenas con un enfoque incremental, garantizando un equilibrio entre **funcionalidad, simplicidad y escalabilidad futura**.

## 2.2 Soluciones de mercado

En el ámbito de la gestión de inventarios y la reposición de productos, existen actualmente diversas soluciones de mercado, desde sistemas ERP de gran escala hasta aplicaciones de inventario más simples. Sin embargo, la mayoría de estas alternativas presentan limitaciones importantes cuando se analizan bajo el contexto de las PYMES chilenas.

### 1. Sistemas ERP (Enterprise Resource Planning)

- **Ejemplos:** SAP Business One, Odoo, Microsoft Dynamics 365.
- **Ventajas:** integración completa de módulos de inventario, contabilidad, finanzas y logística; reportes avanzados en tiempo real; escalabilidad empresarial.
- **Desventajas para PYMES:** altos costos de licencia e implementación, necesidad de conectividad permanente, personal especializado y tiempos de adopción prolongados.

### 2. Software de Inventario especializado

- **Ejemplos:** Zoho Inventory, TradeGecko, Vend.
- **Ventajas:** interfaz intuitiva, módulos específicos de inventario, generación de reportes y compatibilidad con dispositivos móviles.



- **Desventajas:** mayoría requiere suscripción mensual, conexión a internet estable y, en muchos casos, no contempla una lógica de reposición automática con lead time o consumo histórico.

### 3. Soluciones locales y planillas adaptadas

- **Ejemplos:** uso extendido de planillas Excel, macros o software nacional de bajo costo.
- **Ventajas:** flexibilidad, bajo costo inicial, rápida adopción.
- **Desventajas:** elevada probabilidad de errores humanos, ausencia de trazabilidad, falta de validaciones y carencia de módulos de reposición automática.

### Diferenciación de ControlRepro

En comparación con estas alternativas, **ControlRepro** se diferencia por:

- **Autonomía offline:** no depende de conexión a internet, lo que lo hace ideal para PYMES con baja cobertura.
- **Simplicidad:** interfaz clara y validaciones automáticas sin necesidad de capacitación compleja.
- **Foco en la reposición inteligente:** incorpora lead time, stock mínimo y consumo histórico como base del cálculo de pedidos, algo poco común en aplicaciones de bajo costo.
- **Costos accesibles:** elimina la barrera económica de los sistemas ERP internacionales, al ser un software local, liviano y adaptable.
- **Escalabilidad futura:** se proyecta hacia una arquitectura cliente-servidor o en la nube, permitiendo evolución sin perder simplicidad en su versión inicial.

En consecuencia, mientras que las soluciones de mercado actuales tienden a ser **sobredimensionadas para una PYME** o, en el otro extremo, demasiado básicas, **ControlRepro ofrece un equilibrio intermedio**, entregando la funcionalidad crítica de la reposición con un diseño enfocado en **viabilidad técnica, bajo costo y pertinencia local**.

## 2.3 Diseño de alto nivel

El diseño de alto nivel de **ControlRepro** se plantea bajo una arquitectura **modular y escalable**, basada en el patrón **Modelo–Vista–Controlador (MVC)**. Esta elección permite separar de manera clara la lógica de negocio, la interfaz de usuario y la persistencia de datos, facilitando el mantenimiento, la validación incremental y la futura evolución del sistema.

### Estructura general del sistema

#### 1. Capa de Presentación (Vista)



- Desarrollada en **React.js o Vue.js** dentro del entorno **Electron**, lo que permite ofrecer una aplicación multiplataforma.
  - Incluye las interfaces principales: login, dashboard, gestión de productos/proveedores, registro de consumos, pedidos sugeridos y alertas.
  - Se diseñan mockups orientados a garantizar simplicidad y usabilidad, reduciendo la curva de aprendizaje del trabajador.
2. **Capa de Lógica (Controlador / Backend local)**
- Construida en **Node.js con Express**, gestiona las reglas de negocio.
  - Procesa la validación de RUT y PIN, controla la lógica de stock mínimo y lead time, genera pedidos sugeridos y ejecuta la exportación de documentos.
  - Administra la comunicación entre la capa de presentación y la base de datos, asegurando consistencia en la información procesada.
3. **Capa de Datos (Modelo / Persistencia)**
- Basada en **SQLite**, elegida por su simplicidad, rapidez y posibilidad de operar en entornos sin internet.
  - Contiene tablas principales: **trabajadores, productos, proveedores, consumos y pedidos**, todas con integridad referencial garantizada.
  - Permite trazabilidad de las acciones al vincular cada movimiento con el trabajador responsable.

#### Flujos principales del sistema

- **Autenticación:** usuario ingresa credenciales → validación en backend → registro de sesión → acceso a dashboard.
- **Gestión de productos y proveedores:** interfaz CRUD → controlador valida datos → persistencia en SQLite.
- **Registro de consumos:** trabajador ingresa salida de producto → actualización de stock en BD → asociación con responsable.
- **Generación de pedidos sugeridos:** el sistema compara stock actual vs mínimo + consumo histórico + lead time → cálculo de cantidad → creación del pedido.
- **Exportación:** controlador prepara archivo en formato Excel/PDF → guardado automático en escritorio → registro en BD.
- **Alertas:** verificación automática del stock cada vez que se ingresa un consumo o al iniciar sesión → notificación en UI.

#### Pertinencia del diseño

El diseño de alto nivel responde directamente a las necesidades detectadas:

- **Autonomía:** funciona sin conexión a internet.
- **Simplicidad:** interfaz clara para trabajadores sin conocimientos técnicos avanzados.
- **Robustez:** separación en capas asegura mantenibilidad y escalabilidad.

- **Adaptabilidad:** permite evolucionar a futuro hacia una arquitectura cliente-servidor o con despliegue en la nube.

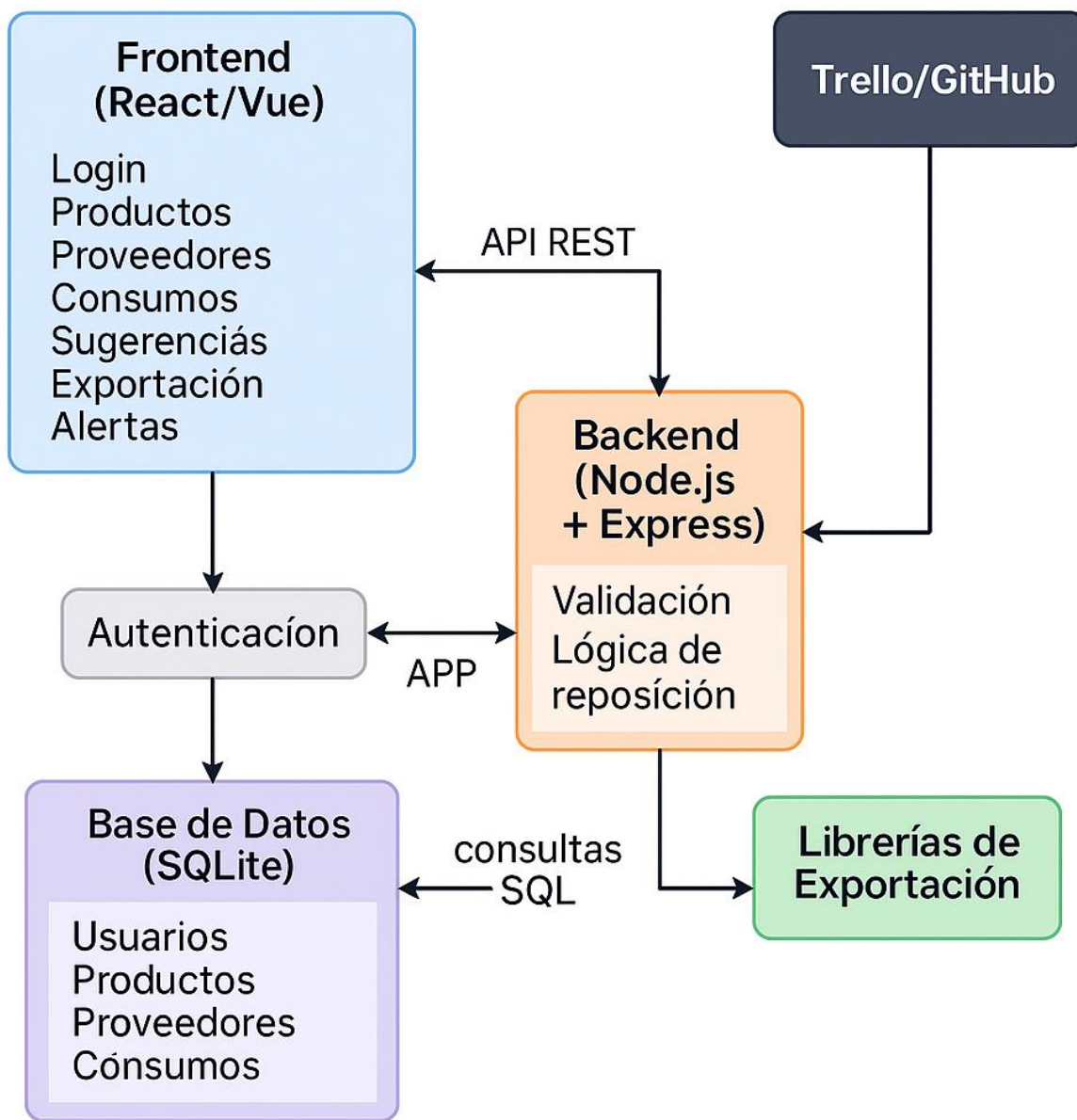


Figura 4. Diseño de alto nivel de ControlRepro (Fuente: Elaboración propia)

El diagrama ilustra la arquitectura modular de **ControlRepro**, organizada bajo el patrón **MVC**. El **frontend** (React/Vue en Electron) concentra las interfaces de usuario y se comunica con el **backend** (Node.js + Express), que gestiona la lógica de reposición y las validaciones. La información se almacena en una **base de datos SQLite**, mientras que las **librerías de exportación** permiten generar pedidos en Excel y PDF. Herramientas externas como **Trello** y **GitHub** apoyan la gestión de tareas y control de versiones.

## 2.4 Factibilidades

El análisis de factibilidad permite evaluar las condiciones necesarias para la implementación de **ControlRepro** en el contexto de las PYMES chilenas. Esta evaluación se realiza desde cuatro dimensiones: técnica, operacional, económica y legal, con el fin de asegurar que la solución propuesta no solo sea funcional, sino también viable y sostenible en su aplicación real.

### 2.4.1 Factibilidad Técnica

- **Arquitectura y tecnología:** ControlRepro está diseñado bajo el patrón **Modelo–Vista–Controlador (MVC)** utilizando **Node.js + Express** en el backend, **React/Vue + Electron** en el frontend, y **SQLite** como base de datos local. Esta elección asegura simplicidad, autonomía y compatibilidad multiplataforma (Windows/Linux).
- **Requisitos mínimos:** el sistema puede ejecutarse en equipos estándar de oficina (CPU i3, 4GB RAM, 500MB libres), evitando inversiones adicionales en infraestructura.
- **Operación offline:** al no depender de conexión a internet, se garantiza continuidad operativa en entornos con baja conectividad.

**Escalabilidad futura:** si bien la versión inicial funciona localmente, la arquitectura está preparada para evolucionar hacia un modelo cliente-servidor o nube (con PostgreSQL/MySQL), lo que asegura su crecimiento a futuro.

La plataforma del MVP se diseñó para operar de manera **local y autónoma**, utilizando SQLite y Electron. No obstante, se deja documentada la posibilidad de una evolución futura hacia un modelo **cliente-servidor o en la nube**, en caso de que empresas con mayor volumen de datos requieran escalabilidad.

**Conclusión técnica:** la propuesta es totalmente factible con los recursos tecnológicos declarados, asegurando funcionamiento estable y la posibilidad de escalar en próximas fases.

### 2.4.2 Factibilidad Operacional

- **Usabilidad:** la interfaz gráfica se desarrolla con principios de simplicidad, validaciones automáticas y mensajes claros, lo que reduce la curva de aprendizaje y facilita la adopción por parte de trabajadores sin formación técnica.
- **Procesos internos:** el sistema reemplaza registros manuales en planillas por una solución digital, mejorando la trazabilidad y reduciendo los errores humanos.
- **Entrenamiento mínimo:** basta con una capacitación inicial corta para trabajadores y administradores, apoyada en el **manual de usuario** y los procedimientos operativos incluidos.
- **Trazabilidad de responsabilidades:** cada acción queda vinculada al trabajador que la ejecuta mediante RUT y PIN, mejorando la transparencia en la operación.

**Conclusión operacional:** el sistema puede ser utilizado de manera eficiente por personal de las PYMES con una capacitación mínima, optimizando procesos sin alterar significativamente las rutinas actuales.

#### 2.4.3 Factibilidad Económica

- **Costos de desarrollo:** el uso de tecnologías open source (Node.js, SQLite, React/Vue, Electron) reduce drásticamente los costos de licenciamiento.
- **Costos de implementación:** no requiere servidores dedicados ni infraestructura compleja; basta con instalar el ejecutable en un computador de oficina.
- **Ahorros proyectados:** al evitar quiebres de stock y reducir pérdidas por errores manuales, se estima un retorno de inversión positivo en corto plazo.
- **Escalabilidad financiera:** las fases futuras (cliente-servidor o nube) podrían implicar costos adicionales, pero se planifican solo para empresas que logren crecimiento y requieran mayor capacidad.

**Conclusión económica:** la propuesta es de **bajo costo y alto impacto**, lo que la hace especialmente viable para el segmento de PYMES.

#### 2.4.4 Factibilidad Legal

- **Protección de datos:** el sistema funciona en modo local, lo que evita riesgos de exposición de información sensible en servidores externos.
- **Normativa chilena:** al registrar trabajadores mediante **RUT**, se asegura consistencia con las normas nacionales de identificación.
- **Licencias de software:** todas las tecnologías utilizadas cuentan con licencias libres o de código abierto, permitiendo su uso sin infracciones legales.
- **Futuras integraciones:** en caso de escalar a la nube, se considerará el cumplimiento con normativas de **protección de datos personales (Ley 19.628 en Chile)** y eventuales exigencias internacionales (como GDPR).

**Conclusión legal:** el sistema cumple con la normativa vigente y no implica riesgos legales en su operación actual.

Tabla 3. Resumen de factibilidades de ControlRepro

| Dimensión          | Aspectos evaluados                                                                                                                                                                                 | Conclusión                                                                    |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| <b>Técnica</b>     | Arquitectura MVC, Node.js + Express, React/Vue + Electron, BD SQLite, operación offline, requisitos mínimos de hardware, posibilidad de escalar a cliente-servidor/nube.                           | Factible con recursos actuales; garantiza estabilidad y escalabilidad futura. |
| <b>Operacional</b> | Interfaz intuitiva, validaciones automáticas, capacitación mínima, reemplazo de planillas, trazabilidad de acciones mediante RUT y PIN.                                                            | Fácil adopción en PYMES, mejora de procesos y reducción de errores humanos.   |
| <b>Económica</b>   | Uso de software open source, bajo costo de implementación, sin necesidad de servidores externos, retorno de inversión rápido gracias a reducción de pérdidas.                                      | Propuesta de bajo costo y alto impacto, viable financieramente para PYMES.    |
| <b>Legal</b>       | Funcionamiento local (sin riesgo de exposición externa), uso de RUT como identificador válido, cumplimiento de Ley 19.628 sobre protección de datos, tecnologías libres con licencias open source. | Cumple normativa vigente, sin riesgos legales actuales ni de licenciamiento.  |

## 2.5 Modelo de Vistas 4+1

El modelo de vistas **4+1** (Kruchten, 1995) se emplea como marco de referencia para representar de manera integral la arquitectura de **ControlRepro**, permitiendo analizar el sistema desde distintas perspectivas que complementan su comprensión y validación.

Las cuatro vistas principales —**lógica, de procesos, de desarrollo (física) y de despliegue**— junto con la vista de escenarios (+1), aseguran que el diseño atienda a los requerimientos funcionales y no funcionales, y que esté alineado con los objetivos del proyecto.

### 2.5.1 Vista Lógica

- **Descripción:** representa la estructura estática del sistema y sus entidades principales.
- **Elementos clave:**
  - **Clases/Entidades:** *Usuario, Producto, Proveedor, Consumo, Pedido.*
  - Relaciones:
    - Un **Proveedor** puede estar asociado a varios **Productos**.
    - Un **Trabajador** (Usuario) registra múltiples **Consumos** y puede generar **Pedidos**.

- Un **Pedido** contiene uno o más **Productos** y está vinculado al trabajador responsable.
- **Objetivo:** asegurar integridad y trazabilidad de los datos que soportan la lógica de negocio.

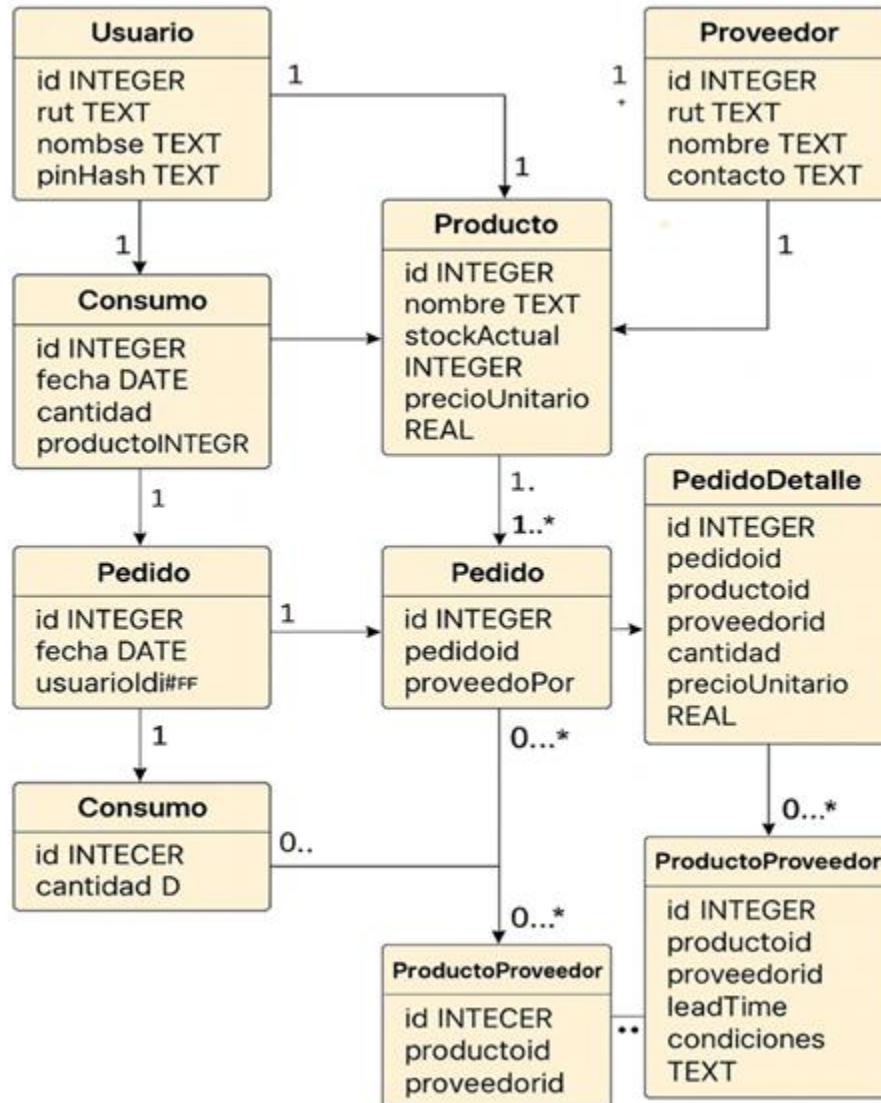


Figura 5 Diagrama de clases (Fuente: Elaboración propia)

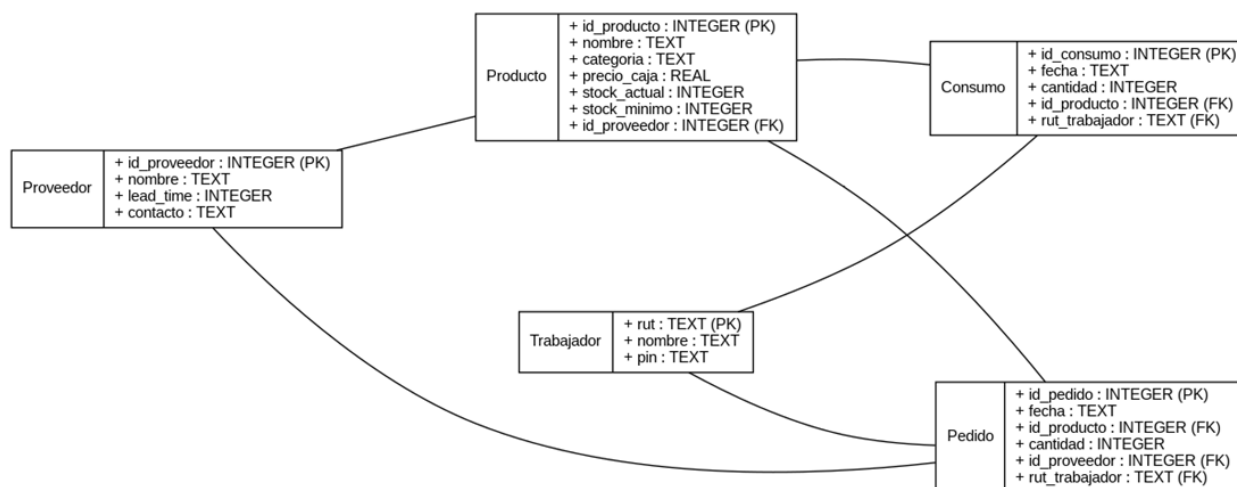


Figura 6. Diagrama de clases simplificado de ControlRepro (Fuente: Elaboración propia).

## 2.5.2 Vista de Despliegue

- **Descripción:** muestra cómo se organiza físicamente el sistema en el entorno de ejecución.
- **Elementos clave:**
  - **Aplicación local de escritorio:** desarrollada en Electron, ejecutable en Windows/Linux.
  - **Base de datos SQLite:** integrada en el equipo del usuario, sin dependencia de servidores externos.
  - **Librerías de exportación:** para generación de Excel y PDF.
  - **Herramientas de soporte:** Trello y GitHub para control de proyecto y versiones.

**Objetivo:** garantizar que el sistema sea autónomo, operativo offline y fácil de instalar en cualquier equipo.

(Para una representación Visual, revisar **Figura 2. Diseño de alto nivel de ControlRepro** (Fuente: Elaboración propia))

## 2.5.3 Vista de Procesos

- **Descripción:** representa el comportamiento dinámico del sistema, mostrando cómo interactúan los componentes durante la ejecución.
- **Flujos clave modelados:**
  1. **Login:** autenticación de trabajador mediante RUT y PIN → validación en backend → acceso autorizado.



2. **Registro de consumo:** trabajador selecciona producto → ingresa cantidad → sistema descuenta stock y guarda registro.
  3. **Generación de pedido:** sistema compara stock actual con stock mínimo y lead time → calcula cantidad → crea pedido sugerido.
  4. **Exportación:** pedido confirmado → backend genera documento Excel/PDF → guardado automático en escritorio.
- **Objetivo:** validar la lógica de interacción y el cumplimiento de los requisitos funcionales

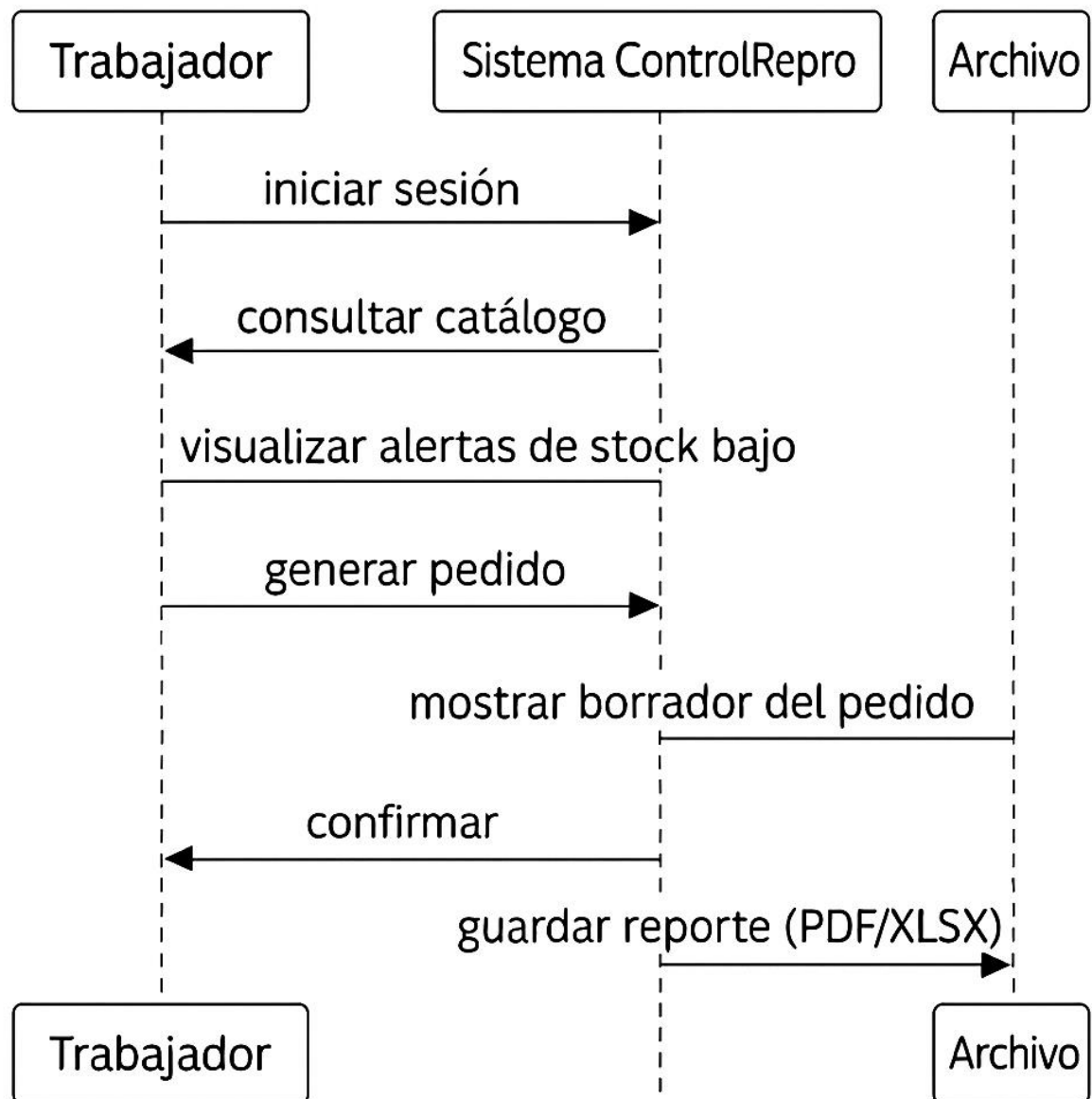


Figura 7 Diagrama de secuencia – generación de pedido (Fuente: Elaboración propia)



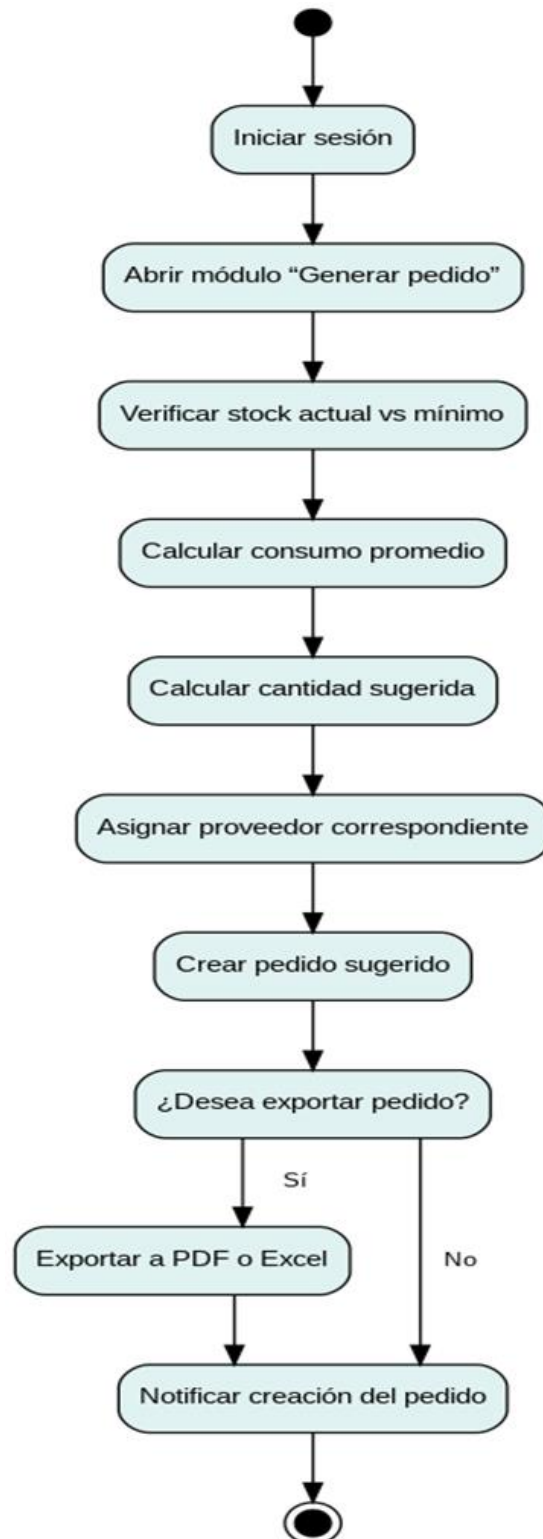


Figura 8. Diagrama de actividades – Flujo de generación de pedidos en ControlRepro (Fuente: Elaboración propia).

#### 2.5.4 Vista Física (Desarrollo)

- **Descripción:** refleja la organización del software desde la perspectiva del desarrollador.
- **Elementos clave:**
  - **Frontend:** construido con React/Vue y Tailwind CSS.
  - **Backend:** Node.js + Express, con controladores y servicios organizados por módulos.
  - **Base de datos:** esquema relacional en SQLite con tablas normalizadas.
  - **Control de versiones:** repositorio GitHub, con ramas para desarrollo y main.
- **Objetivo:** garantizar orden y mantenibilidad del código, facilitando futuras ampliaciones.

#### 2.5.5 Vista de Escenarios (+1)

- **Descripción:** integra las vistas anteriores en casos de uso concretos que validan los requerimientos del sistema.
- **Escenarios críticos:**
  - **Escenario 1:** *Login → Registro de consumo → Generación de pedido → Exportación de pedido.*
  - **Escenario 2:** *Gestión de productos y proveedores → Pedido sugerido con lead time.*
  - **Escenario 3:** *Stock bajo → Alerta automática → Confirmación de pedido.*
- **Objetivo:** demostrar que la arquitectura satisface los requerimientos funcionales y no funcionales definidos en la matriz de trazabilidad.

#### Síntesis del Capítulo II – Fundamentación del tema

El análisis desarrollado en este capítulo permitió establecer las bases conceptuales y técnicas que sustentan el proyecto **ControlRepro**. Se identificó la problemática central de las PYMES en relación con la reposición de inventario, y se presentó una solución tecnológica ajustada a sus necesidades: una aplicación local, simple y autónoma que digitaliza consumos históricos, gestiona productos y proveedores, y genera pedidos sugeridos en función de stock mínimo y lead time.

La comparación con soluciones de mercado permitió evidenciar la pertinencia de ControlRepro, diferenciándose por su accesibilidad, bajo costo, facilidad de uso y autonomía offline. A nivel de diseño, se definió una arquitectura modular bajo el patrón MVC, respaldada por tecnologías de código abierto y un modelo de datos claro que asegura la trazabilidad de las operaciones.

El análisis de factibilidades —técnica, operacional, económica y legal— validó que el sistema no solo es viable en un entorno de desarrollo académico, sino que también responde a condiciones reales de uso en PYMES chilenas. Finalmente, la aplicación del modelo de vistas **4+1** permitió documentar la arquitectura desde distintas perspectivas, demostrando consistencia y coherencia entre requerimientos, diseño y escenarios de uso.

En conjunto, este capítulo confirma que la propuesta es **viable, pertinente y escalable**, estableciendo un marco sólido para el desarrollo metodológico y técnico que se abordará en el **Capítulo III – Materiales y Métodos**.

# CAPÍTULO III MATERIALES Y MÉTODOS

## 3.1 Metodología de gestión del proyecto

La gestión del proyecto **ControlRepro** se fundamenta en el uso de enfoques **ágiles e incrementales**, adaptados a la condición de un equipo reducido (desarrollador único) y a la necesidad de generar resultados verificables semana a semana.

El método seleccionado se basa en una **adaptación de Scrum** y principios de la **metodología incremental**, lo que permite estructurar las actividades en **sprints cortos de una semana** con entregables concretos. Esta elección asegura flexibilidad para incorporar ajustes, control del avance y validación continua de cada módulo desarrollado.

### Principios rectores de la gestión

- **Iteración incremental:** cada semana se entrega un módulo funcional (login, productos, consumos, pedidos, exportación, alertas).
- **Evidencia continua:** cada sprint finaliza con pruebas funcionales y capturas de evidencias que validan el cumplimiento del objetivo semanal.
- **Gestión visual del trabajo:** se utiliza **Trello** como tablero de control, donde cada tarea se registra, se asigna prioridad y se mueve en columnas (*Pendiente, En curso, Completado*).
- **Control de versiones:** el código fuente se administra en **GitHub**, asegurando respaldo y trazabilidad de los cambios en el desarrollo.
- **Gestión de riesgos:** se mantiene una bitácora de errores y desviaciones, registrando las acciones de mitigación aplicadas en cada caso.

### Justificación de la elección

La elección de una metodología ágil responde a la necesidad de:

1. Mantener entregables funcionales en plazos cortos (1 semana por módulo).
2. Reducir la acumulación de errores mediante pruebas incrementales.
3. Facilitar la documentación del avance con base en resultados tangibles.
4. Permitir ajustes rápidos en caso de detectar limitaciones técnicas o de diseño.

En contraste con modelos tradicionales como **cascada**, que exigen una definición rígida y completa antes de la implementación, el enfoque incremental de ControlRepro garantiza una mayor **adaptabilidad** a la realidad del proyecto y una **validación temprana** de los objetivos definidos.

### 3.2 Metodología de desarrollo de software

Inicialmente el proyecto se concibió bajo un enfoque **incremental simple**, pero a medida que avanzó se formalizó en un **marco de trabajo Scrum adaptado**, con sprints semanales y backlog en Trello. Esta evolución metodológica permitió mantener entregables incrementales con una mejor estructura de planificación.

El desarrollo del sistema **ControlRepro** se guía por una **metodología ágil e incremental**, cuyo objetivo es garantizar entregas funcionales de manera progresiva, reduciendo riesgos y asegurando la validación temprana de los módulos que componen el proyecto.

Enfoque metodológico adoptado

- **Modelo incremental:** cada módulo del sistema (login, gestión de productos y proveedores, registro de consumos, motor de sugerencias, exportación y alertas) se desarrolla de manera independiente, entregándose como un bloque funcional validado antes de pasar al siguiente.
- **Iteraciones semanales (sprints):** cada sprint tiene como meta la entrega de un módulo funcional con sus pruebas de validación. Este enfoque asegura que al finalizar cada semana se cuente con un avance verificable, alineado con los hitos del proyecto.
- **Pruebas continuas:** cada módulo pasa por un ciclo de pruebas funcionales (unitarias y de integración parcial) antes de avanzar al siguiente sprint. Esto permite identificar errores en etapas tempranas y corregirlos sin afectar el progreso general.
- **Validación visual y documental:** al cierre de cada sprint se generan evidencias (capturas de pantalla, tablas de comparación y reportes) que se integran al informe como respaldo de cumplimiento.

Ciclo de desarrollo aplicado

El ciclo de desarrollo en ControlRepro sigue la siguiente secuencia:

1. **Definición del módulo:** se describen los objetivos, funcionalidades y criterios de validación.
2. **Diseño técnico:** se elaboran diagramas UML relevantes (casos de uso, clases, secuencia o actividades).
3. **Implementación incremental:** se construye el módulo en el entorno de desarrollo (Node.js, React/Vue, SQLite).
4. **Pruebas funcionales:** se ejecutan validaciones con datos simulados, revisando consistencia de resultados.
5. **Documentación:** se generan capturas de evidencia, registros en la bitácora y actualización en Trello/GitHub.
6. **Liberación de sprint:** el módulo se integra al sistema base y se prepara para el siguiente incremento.

## Beneficios de la metodología

- **Reducción de riesgos:** los errores se detectan en etapas tempranas gracias a la validación incremental.
- **Flexibilidad:** permite realizar ajustes en los módulos siguientes sin afectar lo ya desarrollado.
- **Transparencia del avance:** cada sprint concluye con un entregable verificable que puede evaluarse en términos de funcionalidad real.
- **Documentación continua:** cada iteración deja trazabilidad en informes, bitácoras y repositorios de control de versiones.

## Ejemplo aplicado al proyecto

- **Sprint 1:** Implementación de login seguro (RUT + PIN).
- **Sprint 2:** Gestión de productos y proveedores con lead time.
- **Sprint 3:** Registro de consumos con trazabilidad de trabajador.
- **Sprint 4:** Motor de sugerencias de pedidos.
- **Sprint 5:** Exportación en Excel/PDF.
- **Sprint 6:** Alertas automáticas de stock crítico.

Cada sprint finaliza con la entrega de un **subproducto funcional**, lo que asegura que al término del proyecto (semana 12) se cuente con un **Producto Mínimo Viable (MVP)** plenamente validado.

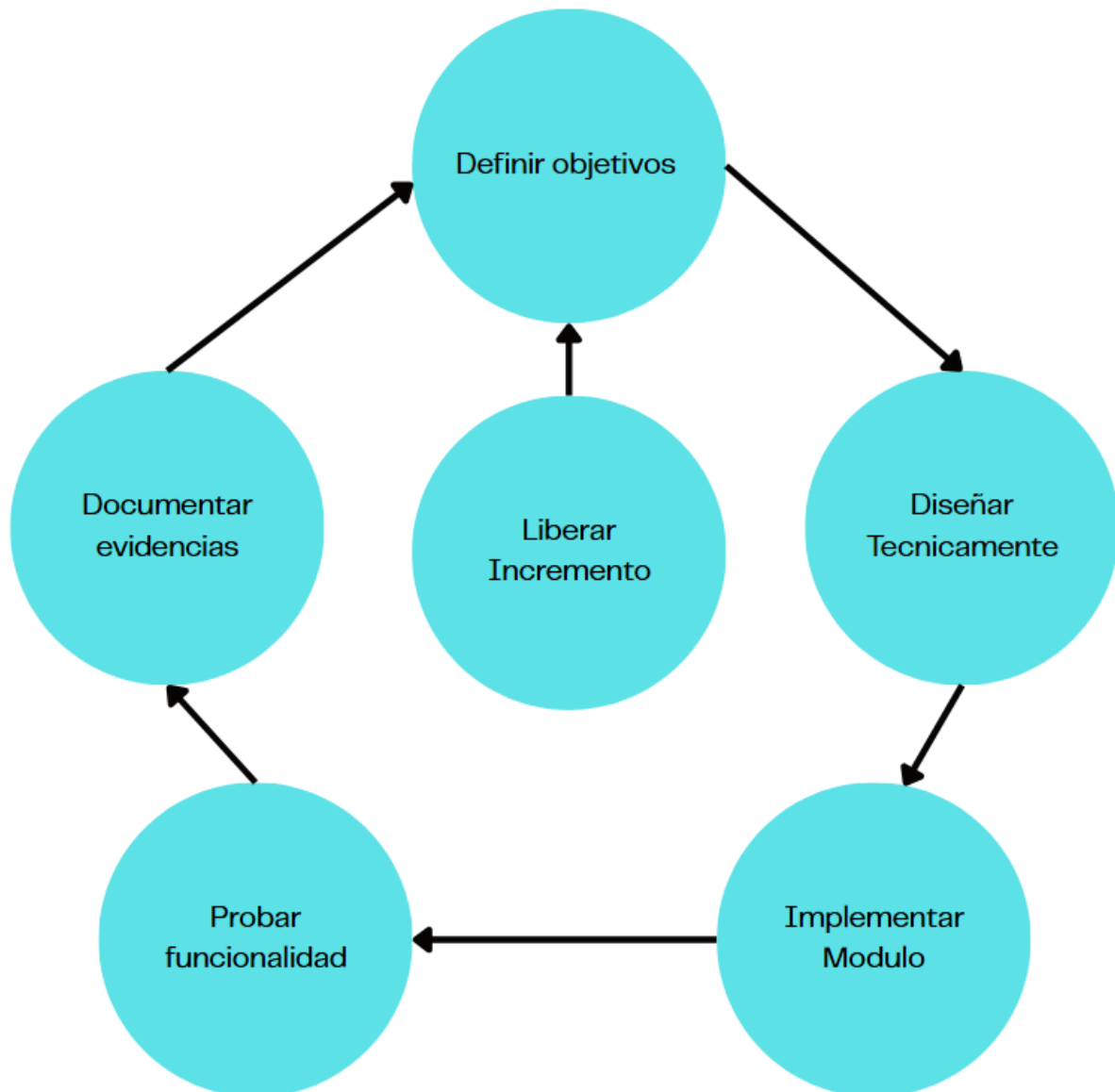


Figura 9. Ciclo de Desarrollo Incremental aplicado a ControlRepro (Fuente: Elaboración propia)

### 3.3 Ambiente de desarrollo y gestión

El ambiente de desarrollo y gestión de **ControlRepro** fue configurado con el objetivo de garantizar un proceso de construcción ordenado, transparente y validable en cada etapa del proyecto. Este ambiente integra tanto las **herramientas técnicas** utilizadas en la programación y pruebas del sistema, como las **herramientas de gestión** orientadas a organizar el trabajo, controlar el avance y mantener la trazabilidad de los resultados.

La elección de este ecosistema responde a criterios de **bajo costo, disponibilidad multiplataforma y facilidad de adopción**, aspectos esenciales para un proyecto orientado a PYMES, donde los recursos suelen ser limitados.

### 3.3.1 Herramientas de desarrollo

El desarrollo de **ControlRepro** se apoya en un conjunto de tecnologías modernas, mayoritariamente **open source**, que aseguran independencia de licencias costosas y flexibilidad para adaptaciones futuras:

- **Visual Studio Code (VS Code):** entorno de desarrollo integrado (IDE) utilizado para programar y depurar el código. Se seleccionó por su ligereza, soporte a múltiples extensiones (Git, SQLite, ESLint) y su capacidad de integrarse fluidamente con Node.js y frameworks frontend.
- **Frontend:** construido en **React.js o Vue.js**, integrados en **Electron** para generar una aplicación de escritorio multiplataforma. Esta capa concentra la interfaz gráfica con el usuario, incluyendo login, dashboard, gestión de productos/proveedores, registro de consumos, pedidos sugeridos y alertas.
- **Backend:** implementado con **Node.js + Express**, encargado de procesar la lógica de negocio (validación de credenciales, control de stock mínimo, cálculo de sugerencias en función de consumo histórico y lead time, exportación de documentos).
- **Base de Datos: SQLite**, un motor relacional embebido y liviano que permite almacenar localmente productos, proveedores, consumos y pedidos. Fue elegido por su autonomía, simplicidad de configuración y por no requerir un servidor dedicado.
- **Estilos visuales: Tailwind CSS**, framework que facilita el diseño de interfaces modernas, responsivas y consistentes con poco esfuerzo de personalización.
- **Exportación de documentos:** librerías **xlsx** y **pdfkit**, que permiten generar automáticamente reportes y pedidos en formatos estándar (Excel y PDF), directamente en el escritorio del usuario.
- **Pruebas funcionales:** se apoyan en datos simulados y validaciones visuales, con evidencias en forma de capturas de pantalla, logs y archivos exportados.

Con esta combinación de herramientas, el proyecto garantiza **simplicidad, autonomía y estandarización**, permitiendo que el producto final sea replicable en cualquier equipo de oficina sin infraestructura adicional.

### 3.3.2 Control de cambios y versiones

La gestión del código fuente se realiza mediante **Git y GitHub**, lo que asegura trazabilidad, seguridad y colaboración estructurada:



- **GitHub:** repositorio central en la nube donde se aloja el código, facilitando el acceso desde cualquier entorno y asegurando un respaldo continuo.
- **Git:** sistema distribuido de control de versiones, que permite trabajar con diferentes ramas de desarrollo y documentar la evolución del código.
- **Estrategia de ramas:**
  - **main:** contiene la versión estable y consolidada del sistema.
  - **dev:** rama de integración para pruebas de nuevas funciones antes de ser promovidas a producción.
  - **feature/:** ramas temporales para el desarrollo de módulos específicos (ejemplo: *feature/login*, *feature/exportación*).

Esta organización garantiza que el desarrollo sea **incremental y seguro**, evitando la pérdida de avances y permitiendo retroceder en caso de errores críticos.

### 3.3.3 Gestión del proyecto y colaboración

La gestión del trabajo y el control del avance se realizan mediante un conjunto de herramientas que apoyan la planificación, la priorización de tareas y la documentación continua:

- **Trello:** tablero Kanban utilizado como sistema de planificación visual. El proyecto se organiza en columnas *Pendiente*, *En curso* y *Completado*, donde cada tarjeta representa una tarea específica (ejemplo: “Implementar login seguro”, “Diseñar base de datos SQLite”).
  - Este método permite **visibilidad inmediata del avance**, facilita la priorización y asegura que cada sprint tenga objetivos claros.
- **Bitácora de errores:** documento auxiliar que registra incidencias detectadas, acciones correctivas y aprendizajes asociados. Esto asegura la trazabilidad de problemas y evita reincidencias.
- **Checklists de control:** al término de cada sprint se utiliza una lista de verificación que valida:
  - (i) diseño correcto del módulo,
  - (ii) implementación funcional,
  - (iii) ejecución de pruebas,
  - (iv) documentación de evidencias.
- **GitHub Projects (integrado al repositorio):** se emplea para vincular issues del código con tareas del desarrollo, reforzando la trazabilidad entre requerimientos y entregables.

### 3.3.4 Beneficios del ambiente configurado

La elección de este ambiente de desarrollo y gestión genera beneficios concretos:

- **Organización y claridad:** cada tarea se encuentra registrada, priorizada y trazada desde su diseño hasta su validación final.
- **Respaldo y seguridad:** GitHub asegura control de versiones, respaldo permanente y posibilidad de trabajar en paralelo con ramas diferenciadas.
- **Flexibilidad y bajo costo:** todas las herramientas son open source o de acceso gratuito, reduciendo barreras económicas.
- **Eficiencia en pruebas:** al incluir procedimientos de validación en cada sprint, se evitan errores acumulativos y se asegura calidad en los incrementos.
- **Documentación continua:** Trello, GitHub y la bitácora de errores permiten mantener un registro permanente del avance y decisiones tomadas.

### 3.4 Scrum aplicado al proyecto

El desarrollo de **ControlRepro** se organizó siguiendo un marco de trabajo **ágil adaptado a Scrum**, adecuado para un equipo reducido de desarrollo (un desarrollador principal). Esta adaptación permitió estructurar el trabajo en **sprints semanales**, con entregables funcionales en cada iteración, asegurando control del avance, reducción de riesgos y validación temprana de resultados.

Si bien Scrum se diseñó originalmente para equipos multidisciplinarios, en este proyecto se adoptó como **guía metodológica**, ajustando la definición de roles y eventos a las condiciones reales del proyecto.

#### 3.4.1 Roles del proyecto

En la implementación de Scrum dentro de ControlRepro, los roles tradicionales se reinterpretaron de la siguiente forma:

- **Product Owner (PO):** asumido por el mismo desarrollador, responsable de priorizar los objetivos del sistema (login, productos/proveedores, consumos, pedidos, exportaciones, alertas) y de asegurar que el producto cumpla con los requerimientos planteados.
- **Scrum Master (SM):** rol también asumido por el desarrollador, encargado de mantener la disciplina metodológica, registrar la bitácora de errores, gestionar las reuniones de control y asegurar que se respeten los tiempos de cada sprint.
- **Development Team:** en este caso el equipo de desarrollo estuvo compuesto por un único integrante, lo que simplifica la gestión de dependencias, pero mantiene la exigencia de cumplir con los entregables semanales definidos.

Esta adaptación de roles permitió conservar la **filosofía de Scrum**, a pesar de la escala reducida del proyecto.

### 3.4.2 Procesos y eventos del proyecto

El ciclo de trabajo en cada sprint incluyó los siguientes procesos:

- **Sprint Planning:** al inicio de cada semana se definían las tareas específicas a desarrollar y los criterios de aceptación. Esto se registraba en **Trello** como tarjetas en la columna *Pendiente*.
- **Daily Scrum:** dado que el equipo fue unipersonal, la reunión diaria se tradujo en un **registro breve en la bitácora** sobre los avances, problemas encontrados y ajustes realizados.
- **Sprint Review:** al cierre de cada sprint se validaban las funcionalidades implementadas mediante **pruebas funcionales** y se generaban capturas de evidencia.
- **Sprint Retrospective:** después de cada revisión, se documentaban en la bitácora las lecciones aprendidas, errores comunes y recomendaciones para el siguiente sprint.

### 3.4.3 Definición del sprint

Cada sprint tuvo una duración de **una semana**, con el objetivo de entregar un módulo funcional validado:

- **Sprint 1:** Login seguro (validación de RUT y PIN).
- **Sprint 2:** Gestión de productos y proveedores con lead time.
- **Sprint 3:** Registro de consumos históricos y control de stock.
- **Sprint 4:** Motor de sugerencias de pedidos.
- **Sprint 5:** Exportación a Excel/PDF.
- **Sprint 6:** Sistema de alertas automáticas de stock crítico.

De esta manera, al finalizar el ciclo de seis sprints, se obtuvo un **Producto Mínimo Viable (MVP)** completo, validado funcionalmente y con evidencias documentadas.

### 3.4.4 Definición de prioridad

La priorización de tareas se basó en el **impacto sobre el objetivo general** y la **dependencia lógica entre módulos**:

1. Funcionalidades críticas para la operación del sistema (**login, productos y consumos**) tuvieron la mayor prioridad.
2. Funcionalidades derivadas (**sugerencias de pedido, exportaciones**) se desarrollaron en etapas intermedias.
3. Funcionalidades complementarias (**alertas automáticas**) se implementaron en las últimas fases, tras consolidar la base del sistema.

Esta jerarquización garantizó que, incluso en caso de restricciones de tiempo, el sistema conservara un núcleo funcional válido.

### 3.4.5 Definición de puntos de esfuerzo

Cada tarea se evaluó en términos de **puntos de esfuerzo**, considerando la complejidad técnica y el tiempo estimado:

- **1 punto:** tareas simples de diseño o validación (ejemplo: ajustar estilos de interfaz, crear tabla en BD).
- **2 puntos:** tareas de complejidad media (ejemplo: CRUD de productos, validación de stock).
- **3 puntos:** tareas críticas con múltiples dependencias (ejemplo: motor de sugerencias de pedido, exportación en PDF).

Este esquema permitió medir el avance semanal de manera cuantitativa y ajustar la carga de trabajo a las 12 semanas de desarrollo definidas como plazo del proyecto.

### 3.4.6 Product Backlog

El **Product Backlog** de ControlRepro se organizó en **Trello**, estructurado en tarjetas que representaban cada funcionalidad, prueba o tarea complementaria. Entre los ítems más relevantes se encuentran:

- **Login:** validación de RUT, validación de PIN, registro de sesiones.
- **Productos/Proveedores:** CRUD productos, CRUD proveedores, asociación proveedor-producto, gestión de lead time.
- **Consumos:** registro de consumos, vinculación a trabajador, control de stock mínimo.
- **Pedidos:** generación de sugerencias, vista previa, confirmación de pedido.
- **Exportación:** generación de Excel, generación de PDF, guardado en escritorio.
- **Alertas:** verificación de stock crítico, visualización en dashboard, tabla de alertas internas.
- **Pruebas:** pruebas unitarias por módulo, pruebas de integración, pruebas de aceptación.

Este backlog se fue refinando semana a semana, incorporando tareas derivadas de la bitácora de errores y ajustes solicitados durante el desarrollo.

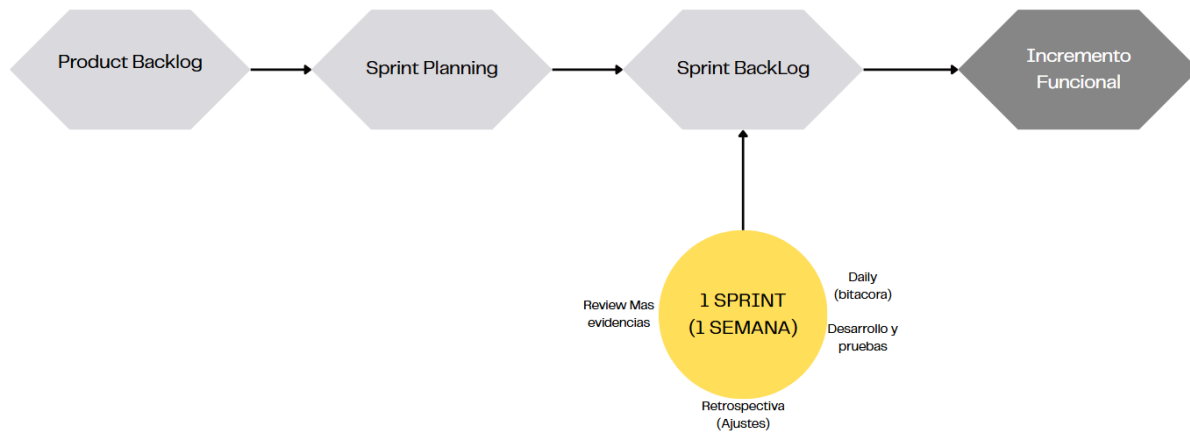


Figura 10. Esquema de Scrum adaptado a ControlRepro (Fuente: Elaboración propia)

### 3.5 Planificación del proyecto

La planificación de **ControlRepro** se estructura en **12 semanas** bajo un esquema **ágil-incremental con sprints semanales**. El objetivo es asegurar entregables funcionales continuos, evidencias verificables y un cierre con pruebas integrales y documentación final.

#### 3.5.1 Cronograma macro (12 semanas)

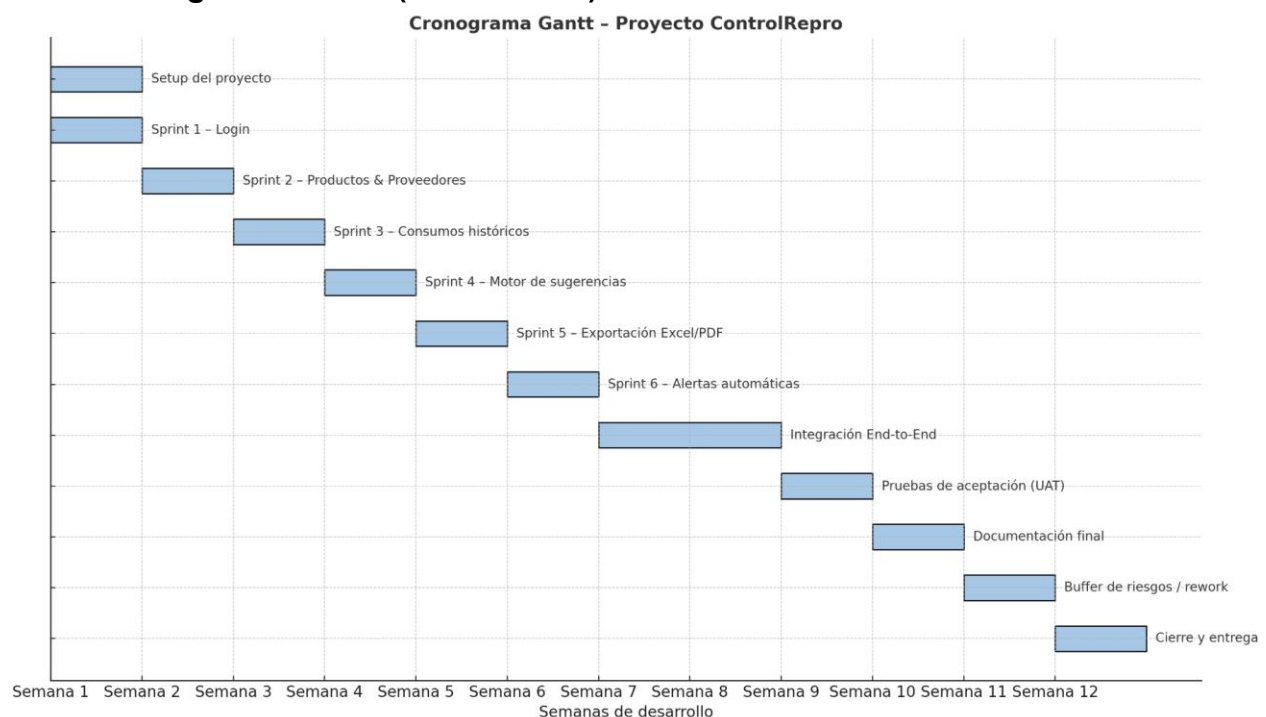


Figura 11. Cronograma Gantt de planificación del proyecto ControlRepro (Fuente: Elaboración propia).

### 3.5.2 Hitos y entregables

| Hito                 | Semana | Entregable tangible                                             | Evidencias asociadas                         |
|----------------------|--------|-----------------------------------------------------------------|----------------------------------------------|
| H0 – Línea base      | 1      | Repositorio GitHub, estructura Electron/Node, BD SQLite inicial | Capturas VSCode, README, esquema BD vacío    |
| H1 – Login operativo | 1      | Autenticación RUT+PIN, control de sesión                        | Casos de prueba PF-01, logs de acceso        |
| H2 – Maestros        | 2      | CRUD Productos/Proveedores + lead time                          | PF-02/03, capturas CRUD, datos de ejemplo    |
| H3 – Consumos        | 3      | Registro de consumos con trazabilidad                           | PF-04, tabla consumos con RUT responsable    |
| H4 – Sugerencias     | 4      | Cálculo de pedido sugerido                                      | PF-05, reporte de precisión con escenarios   |
| H5 – Exportación     | 5      | Exportación a Excel/PDF en escritorio                           | Archivos generados, nomenclatura estándar    |
| H6 – Alertas         | 6      | Alertas por stock crítico en dashboard                          | Capturas UI, tabla de alertas                |
| H7 – Integración E2E | 7–8    | Flujo completo<br>Login→Consumo→Pedido→Exportación              | Video corto/serie de capturas, checklist E2E |
| H8 – UAT             | 9      | Pruebas de aceptación, criterios aprobados                      | Acta UAT, lista de issues cerrados           |
| H9 – Documentación   | 10     | Manual usuario, instalación y PO                                | PDFs/Docx finales                            |
| H10 – Cierre         | 12     | Build instalable + informe final                                | Ejecutable, índice final, anexos             |

### 3.5.3 Plan de sprints (detallado)

#### Sprint 1 – Login (RUT + PIN)

- **Objetivo:** acceso seguro y trazable.
- **Historias:** “Como trabajador, quiero iniciar sesión con RUT y PIN para registrar mis acciones”.
- **Tareas clave:** validación de formato RUT, hash/almacenamiento de PIN, pantalla de login, manejo de sesión.
- **Criterios de aceptación:** credenciales válidas permiten acceso; inválidas muestran error; registro de inicio/fin de sesión.
- **Evidencias:** PF-01, capturas login, logs de sesión.

#### Sprint 2 – Productos & Proveedores

- **Objetivo:** datos maestros con lead time.
- **Historias:** “Como admin, quiero gestionar productos y proveedores para planificar reposición”.
- **Tareas:** CRUD productos, CRUD proveedores, relación producto–proveedor, campo lead time por proveedor.
- **Criterios:** altas/bajas/modificaciones persistentes; validaciones de duplicidad; lead time visible en producto.
- **Evidencias:** PF-02/03, capturas de formularios y listados.

#### Sprint 3 – Consumos históricos

- **Objetivo:** historial confiable con responsable.
- **Historias:** “Como trabajador, registro consumos para mantener actualizado el stock”.
- **Tareas:** formulario consumo, actualización stock, FK a usuario, reportes básicos.
- **Criterios:** no permite negativos; registra fecha/usuario; refleja stock actualizado.
- **Evidencias:** PF-04, tabla consumos, mockup reporte simple.

Nota: En esta versión funcional no se incluyó la asociación automática al usuario, ya que corresponde a un requerimiento no funcional pendiente. PF-04.3

#### Sprint 4 – Motor de sugerencias

- **Objetivo:** pedido sugerido usando stock mínimo + consumos + lead time.
- **Historias:** “Como admin, quiero sugerencias automáticas para evitar quiebres”.
- **Tareas:** servicio de cálculo, regla base, vista de pedido sugerido, confirmación.
- **Criterios:** genera cantidad  $\geq 0$ ; respeta stock mínimo; usa lead time del proveedor; reporte de pruebas con  $\geq 3$  escenarios.
- **Evidencias:** PF-05, planillas de prueba, capturas.

### Sprint 5 – Exportación Excel/PDF

- **Objetivo:** documentos listos para envío.
- **Historias:** “Como admin, quiero exportar pedidos a Excel/PDF con firma/responsable”.
- **Tareas:** plantilla XLSX/PDF, guardado en escritorio, nomenclatura pedido\_RUT\_fecha.
- **Criterios:** archivos se abren correctamente; contienen todos los campos; ruta y nombre correctos.
- **Evidencias:** archivos generados, PF-06.

### Sprint 6 – Alertas automáticas

- **Objetivo:** aviso oportuno de stock crítico.
- **Historias:** “Como usuario, quiero alertas cuando el stock  $\leq$  mínimo”.
- **Tareas:** servicio de verificación, badge en dashboard, listado de alertas.
- **Criterios:** alerta se genera al registrar consumo y/o al iniciar sesión; queda persistida.
- **Evidencias:** capturas dashboard, tabla alertas, PF-07.

### Semanas 7–9 – Integración, endurecimiento y UAT

- **Objetivo:** flujo E2E robusto y aceptado.
- **Tareas:** pruebas cruzadas entre módulos, cierre de issues, UAT con criterios formales.
- **Criterios:** 100% de casos críticos pasados; 0 defectos bloqueantes.
- **Evidencias:** checklist E2E, acta UAT.

### Semana 10 – Documentación

- Manual de usuario, instalación y procedimientos operativos (PO).

### Semana 11 – Buffer

- Rework controlado (correcciones de UAT, performance, pequeños ajustes).

### Semana 12 – Cierre

- Build final, empaquetado, anexos y entrega.

### 3.5.4 Dependencias principales

- **Login** es prerequisite para **Consumos** y **trazabilidad**.
- **Productos/Proveedores** es prerequisite para **Consumos**, **Sugerencias**, **Exportación** y **Alertas**.
- **Sugerencias** depende de **Consumos** y **Maestros**.
- **Exportación** depende de **Sugerencias**.
- **Alertas** depende de **Consumos** y **stock mínimo**.



### 3.5.5 Métricas de control

- **Velocidad objetivo:** 6–9 puntos por sprint (según complejidad).
- **Defectos bloqueantes por sprint:**  $\leq 0$ ; **defectos mayores:**  $\leq 2$  (resueltos antes del siguiente sprint).
- **Cumplimiento de criterios de aceptación:**  $\geq 95\%$ .
- **Tiempo de ciclo por historia clave:**  $\leq 5$  días hábiles.
- **Cobertura de pruebas funcionales (manuales):** 100% de historias “must”.

### 3.5.6 Calendario de ceremonias (adaptado)

- **Sprint Planning:** lunes (30–45 min).
- **Daily (bitácora):** registro breve diario de avances/impedimentos.
- **Review:** viernes (pruebas y evidencias).
- **Retrospectiva:** viernes (15–20 min, acciones de mejora al backlog).

### 3.5.7 Gestión de buffers y contingencias

- **Micro-buffer por sprint:** 10% del tiempo para imprevistos.
- **Buffer global: Semana 11** destinada a correcciones resultantes de UAT y ajustes finales.
- **Criterio de escalamiento:** si una historia crítica se atrasa  $>2$  días, se reduce alcance de historias “should” del sprint o se traslada a sprint siguiente manteniendo el **núcleo del MVP** intacto.

## 3.6 Plan de pruebas

El plan de pruebas de **ControlRepro** asegura la verificación sistemática de los requerimientos del MVP (login, maestros, consumos, sugerencias, exportación y alertas) mediante **pruebas funcionales, de integración (end-to-end) y de aceptación (UAT)**. Se ejecuta por sprint (validación incremental) y vuelve a correrse en una pasada final de **regresión** antes de la entrega.

### 3.6.1 Alcance y objetivos

- **Alcance:** funcionalidades del MVP operando en modo **offline** sobre un equipo estándar (Windows/Linux) con **SQLite** local.
- **Objetivos:**
  1. Comprobar que cada requisito funcional se cumple con criterios medibles.
  2. Validar los flujos **extremo a extremo** (Login→Consumos→Sugerencia→Exportación→Alertas).
  3. Aceptar formalmente el producto con criterios predefinidos (UAT).

### 3.6.2 Estrategia y niveles

1. **Pruebas funcionales por módulo (PF)** – se ejecutan en el sprint que entrega el módulo.
2. **Pruebas de integración (PI/E2E)** – se ejecutan en sprints 7–8 y antes del cierre.
3. **Pruebas de aceptación (UAT)** – se ejecutan en la semana 9 con escenarios de usuario y checklist de criterios.
4. **Regresión** – re-ejecución abreviada de los casos críticos tras correcciones (semanas 10–11).

### 3.6.3 Entorno de pruebas

- **SO:** Windows 10/11 o Ubuntu 22.04.
- **HW mínimo:** CPU i3, 4 GB RAM, 500 MB libres.
- **BD:** SQLite (archivo local).
- **Datos semilla:**
  - 1 usuario válido (RUT realista y PIN).
  - 10 productos, 3 proveedores, con **stock mínimo** y **lead time** por proveedor.
  - ≥10 consumos simulados con fechas recientes (últimos 60 días).
- **Artefactos:** build instalable, planillas de casos, checklist UAT, bitácora de defectos.

### 3.6.4 Diseño de casos de prueba funcional (PF)

Nota: IDs PF-\* se referencian en la matriz de trazabilidad (sección 1.4.3).

| ID      | Módulo      | Caso de prueba                    | Datos de entrada                            | Resultado esperado (criterios de aceptación)                                |
|---------|-------------|-----------------------------------|---------------------------------------------|-----------------------------------------------------------------------------|
| PF-01   | Login       | Autenticación RUT + PIN válidos   | RUT formato 12.345.678-9, PIN correcto      | Acceso concedido, registro de sesión con timestamp y RUT.                   |
| PF-01.1 | Login       | RUT malformado / PIN inválido     | RUT "12345678-9", PIN erróneo               | Bloqueo de acceso, mensaje de error legible, sin sesión activa.             |
| PF-02   | Productos   | Crear/editar/eliminar producto    | Nombre, precio, stock mínimo                | Persistencia correcta, validación de duplicados, stock mínimo ≥0.           |
| PF-03   | Proveedores | CRUD proveedor + lead time        | Nombre, lead time (días)                    | Persistencia correcta, lead time numérico positivo, relación con productos. |
| PF-04   | Consumos    | Registrar consumo con responsable | Producto X, cantidad>0, usuario autenticado | Descuento de stock, registro con fecha y RUT; impedir stock negativo.       |

| ID      | Módulo      | Caso de prueba                      | Datos de entrada                      | Resultado esperado (criterios de aceptación)                                                                       |
|---------|-------------|-------------------------------------|---------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| PF-05   | Sugerencias | Generar pedido sugerido             | Maestro completo, consumos existentes | Cantidades sugeridas calculadas con <b>stock mínimo + consumo histórico + lead time</b> ; vista previa del pedido. |
| PF-05.1 | Sugerencias | Escenario sin datos suficientes     | Producto sin consumos                 | Mensaje de advertencia y comportamiento por defecto documentado.                                                   |
| PF-06   | Exportación | Exportar pedido a Excel             | Pedido confirmado                     | Archivo .xlsx en Escritorio: pedido_<RUT>_<AAAA-MM-DD>.xlsx; contenido: items, cantidades, proveedor, responsable. |
| PF-06.1 | Exportación | Exportar pedido a PDF               | Pedido confirmado                     | Archivo .pdf con la misma estructura; apertura correcta.                                                           |
| PF-07   | Alertas     | Disparo de alerta por stock crítico | Stock actual ≤ mínimo                 | Badge/entrada en tabla de alertas, persistencia del evento y visualización en dashboard.                           |

### 3.6.5 Pruebas de integración / end-to-end (PI)

| ID    | Flujo E2E                | Pasos                                               | Resultado esperado                                                          |
|-------|--------------------------|-----------------------------------------------------|-----------------------------------------------------------------------------|
| PI-01 | Login → Consumo          | Iniciar sesión → Registrar consumo de Producto A    | Stock actualizado, consumo asociado a RUT; sin errores de concurrencia.     |
| PI-02 | Consumo → Sugerencia     | Registrar varios consumos → Ejecutar motor          | Pedido sugerido refleja consumos y lead time; cantidades coherentes.        |
| PI-03 | Sugerencia → Exportación | Confirmar pedido → Exportar Excel y PDF             | Se generan ambos archivos, con nomenclatura y datos correctos.              |
| PI-04 | Consumo → Alerta         | Consumir hasta superar umbral                       | Nueva alerta visible y persistida; estado se mantiene tras reinicio de app. |
| PI-05 | Flujo completo           | Login → Consumo → Sugerencia → Exportación → Alerta | Sin fallos; tiempos de respuesta aceptables (<2 s por operación local).     |

### 3.6.6 Pruebas de aceptación (UAT)

- **Participantes:** usuario tipo *administrador de bodega* (rol de negocio).

- **Escenarios UAT:**
  - **UAT-01:** Generar un pedido sugerido completo a partir de consumos reales simulados.
  - **UAT-02:** Exportar el pedido y validar que el archivo sea legible y completo.
  - **UAT-03:** Forzar un stock crítico y verificar la alerta.
- **Criterios de aceptación globales:**
  - 100% de **PF** críticos y **PI** superados.
  - **0 defectos bloqueantes**,  $\leq 2$  defectos mayores abiertos (con plan de corrección).
  - Cumplimiento de **nomenclatura de archivos y trazabilidad por RUT**.
  - Tiempo total para flujo UAT-01  $\leq 2$  min desde login a exportación.

### 3.6.7 Gestión de defectos

- **Severidad:** Bloqueante (impide operación), Mayor (afecta resultado), Menor (cosmético/UX).
- **Ciclo:** Abrir issue → Reproducir → Corregir → Verificar → Cerrar.
- **Métricas:** tasa de cierre semanal, defectos por módulo, re-aperturas.
- **Herramientas:** bitácora de errores + GitHub Issues.

### 3.6.8 Criterios de salida por nivel

- **Funcional (PF):** todos los casos del módulo **OK**; sin bloqueantes; documentación y capturas subidas.
- **Integración (PI):** PI-01...PI-05 **OK**; estabilidad del build  $\geq 2$  horas de operación continua.
- **UAT:** checklist de aceptación firmado; sin bloqueantes; plan de acción para menores.
- **Regresión final:** re-ejecución de PF críticos y PI-05 tras correcciones (todo **OK**).

### 3.6.9 Evidencias y trazabilidad

- **Evidencias:** capturas de pantalla, archivos exportados (XLSX/PDF), logs de sesión y de alertas.
- **Trazabilidad:** cada caso referencia su requisito **RF/RNF** y el **sprint** donde se validó, manteniendo alineación con la **matriz de trazabilidad** del capítulo I.

**Tabla                      modelo                      para                      registrar                      resultados**

| ID Caso | Fecha      | Sprint | Resultado (OK/FAIL) | Defecto # | Observaciones / Evidencia                   |
|---------|------------|--------|---------------------|-----------|---------------------------------------------|
| PF-05   | 2025-MM-DD | 4      | OK                  | –         | Captura pedido_RUT_2025-..xlsx adjunta      |
| PI-03   | 2025-MM-DD | 8      | FAIL                | #34       | PDF sin logo; reprobado, corregir plantilla |

### 3.7 Plan de riesgos

El desarrollo de **ControlRepro** considera riesgos tanto técnicos como operacionales, económicos y de gestión. La gestión de riesgos se estructura en tres fases:

1. **Identificación:** listado exhaustivo de riesgos potenciales.
2. **Evaluación:** análisis de probabilidad (Baja, Media, Alta) e impacto (Leve, Medio, Crítico).
3. **Respuesta:** definición de medidas preventivas y planes de contingencia.

La finalidad es reducir la probabilidad de ocurrencia y mitigar los efectos negativos que pudieran comprometer la entrega del proyecto.

#### 3.7.1 Identificación de los riesgos

Los riesgos detectados se clasifican en **técnicos**, **operacionales**, **económicos** y **de gestión**:

- **Técnicos:**
  - Fallas en la integración de módulos (ej. exportación → BD).
  - Incompatibilidad de librerías en build final (Electron/Node).
  - Pérdida de datos por mal manejo de SQLite.
- **Operacionales:**
  - Errores del usuario final en la digitación de datos (RUT mal formateado, consumos mal registrados).
  - Baja adopción por parte de trabajadores sin experiencia digital.
- **Económicos:**
  - Sobrecarga de horas de desarrollo frente al plan inicial.
  - Posibles costos de librerías externas en fases futuras (ej. migración a nube).
- **De gestión:**
  - Retrasos en los sprints por sobreestimación de tareas.
  - Escasa evidencia documentada que dificulte validación final.

### 3.7.2 Evaluación de los riesgos

| ID | Riesgo                                                  | Probabilidad | Impacto | Nivel | Justificación                                                  |
|----|---------------------------------------------------------|--------------|---------|-------|----------------------------------------------------------------|
| R1 | Fallo en integración de módulos                         | Media        | Crítico | Alto  | Dependencia entre motor de sugerencias y exportación.          |
| R2 | Incompatibilidad de librerías en empaquetado (Electron) | Media        | Medio   | Medio | Cambios de versión pueden afectar compatibilidad.              |
| R3 | Pérdida de datos en SQLite                              | Baja         | Crítico | Medio | Mal manejo de registros podría corromper la BD.                |
| R4 | Errores de usuario en digitación                        | Alta         | Medio   | Alto  | RUT mal escrito o consumo mal ingresado impacta en stock.      |
| R5 | Baja adopción de la herramienta                         | Media        | Medio   | Medio | Resistencia al cambio en PYMES poco digitalizadas.             |
| R6 | Sobrecarga de horas de desarrollo                       | Media        | Crítico | Alto  | Riesgo por único desarrollador y plazos acotados.              |
| R7 | Costos futuros en librerías/nube                        | Baja         | Medio   | Bajo  | Aplica solo en escalabilidad.                                  |
| R8 | Retraso en sprints                                      | Media        | Crítico | Alto  | Cada sprint es semanal y depende de entregables incrementales. |
| R9 | Falta de evidencia documentada                          | Media        | Medio   | Medio | La rúbrica exige pruebas claras y anexos.                      |

### 3.7.3 Plan de respuesta y contingencia

| ID | Estrategia de respuesta | Medidas preventivas                                     | Contingencia                                                       |
|----|-------------------------|---------------------------------------------------------|--------------------------------------------------------------------|
| R1 | Mitigar                 | Pruebas unitarias e integración progresiva por sprint   | Refactorizar dependencias y priorizar exportación mínima funcional |
| R2 | Mitigar                 | Fijar versiones de Node/Electron, testear builds en dev | Cambiar librería/exportación manual como respaldo                  |
| R3 | Prevenir                | Backups automáticos de SQLite en cada cierre de app     | Restaurar desde backup y validar integridad                        |

| ID | Estrategia de respuesta | Medidas preventivas                                         | Contingencia                                        |
|----|-------------------------|-------------------------------------------------------------|-----------------------------------------------------|
| R4 | Mitigar                 | Validación de RUT y cantidades con mensajes de error claros | Capacitación breve de usuario final                 |
| R5 | Mitigar                 | Diseñar interfaz simple, guías visuales y manual de usuario | Taller inicial de uso para trabajadores             |
| R6 | Transferir / Mitigar    | Definir puntos de esfuerzo realistas, micro-buffer semanal  | Reasignar prioridades, postergar extras no críticos |
| R7 | Aceptar                 | Planificar migración solo si se justifica                   | Usar alternativas open source en caso de costos     |
| R8 | Mitigar                 | Planificación realista con backlog priorizado               | Reagendar sprint, usar buffer de semana 11          |
| R9 | Prevenir                | Capturas y logs de cada prueba desde sprint 1               | Documentar evidencias faltantes en anexos           |

### 3.7.4 Síntesis

La gestión de riesgos en **ControlRepro** se apoya en un enfoque **proactivo y adaptativo**, que combina prevención, mitigación y contingencias claras. La existencia de un **buffer de riesgos en la semana 11** asegura margen de maniobra para absorber retrasos o resolver fallas críticas sin comprometer la entrega del MVP.

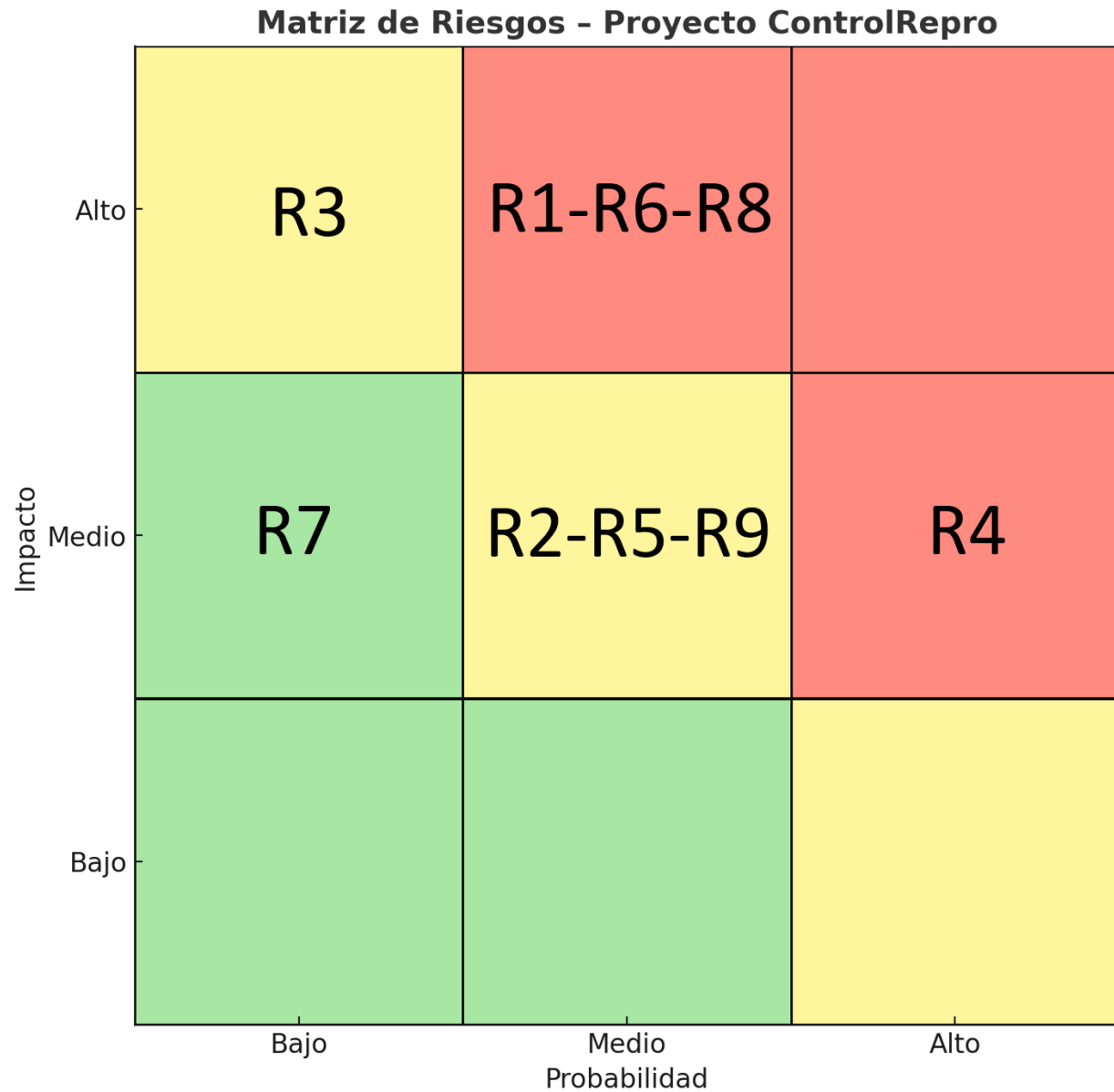


Figura 12. Matriz de Riesgos del proyecto ControlRepro (Fuente: Elaboración propia).



# CAPÍTULO IV GESTIÓN DEL DESARROLLO DEL SISTEMA

## 4.1 Sprint 1 – Login (RUT + PIN)

### 4.1.1 Planificación

El primer sprint se enfocó en implementar el **módulo de autenticación**, elemento crítico para la trazabilidad del sistema, ya que todas las operaciones de consumo y pedidos deben estar vinculadas a un trabajador identificado.

- **Objetivo:** desarrollar un login seguro basado en **RUT y PIN**.
- **Duración:** 1 semana.
- **Requerimientos asociados:**
  - **RF-01:** Validar RUT y PIN al iniciar sesión.
  - **RNF-01:** Aplicación debe funcionar sin internet (login offline en SQLite).
  - **RNF-02:** Registrar usuario responsable en cada acción.
- **Criterios de aceptación:**
  - Ingreso correcto concede acceso a la aplicación.
  - Credenciales inválidas generan mensaje de error claro.
  - Se registra en logs el inicio y fin de sesión con timestamp y RUT.

### 4.1.2 Diseño de vistas

- **Pantalla de login:** formulario con dos campos:
  - **RUT:** con validación de formato chileno (ej. 12.345.678-9).
  - **PIN:** numérico de 4–6 dígitos.
- **Controles implementados:**
  - Validación automática de formato de RUT.
  - Botón *Ingresar* que verifica credenciales en SQLite.
  - Mensaje emergente en caso de error



Figura 13. Pantalla de Login de ControlRepro (Fuente: Captura del prototipo funcional).

#### 4.1.3 Riesgos

- **Errores de formato en el RUT** → mitigado con validación automática.
- **PIN olvidado o incorrecto** → mitigado mediante opción de restablecimiento por administrador en BD.
- **Fallos en SQLite (archivo corrupto)** → mitigado con copias de respaldo al iniciar la app.

#### 4.1.4 Evidencia de liberación del producto

Al término del sprint, se entregó un **login funcional operativo en local**, con autenticación validada en la base de datos.

- Usuarios de prueba cargados:
  - RUT: 12.345.678-9, PIN: 1234.
  - RUT: 20.111.333-5, PIN: 5678.

- Resultado: acceso exitoso para credenciales válidas; error controlado en caso de credenciales inválidas.



The screenshot shows the login interface of the ControlRepro application. At the top is the 'CONTROL REPRO' logo. Below it is the title 'Inicio de Sesión'. There are two input fields: the first contains the RUT '11.111.111-2' and the second contains masked characters '....'. A blue 'Ingresar' button is positioned below the fields. A red warning message with a triangle icon states '⚠ RUT o PIN incorrectos'.

figura 14. inicio de sesión fallido en la aplicación de ControlRepro

#### 4.1.5 Evidencia del ciclo de desarrollo del Sprint 1

- **Planificación:** tarjeta en Trello con tareas *Pendiente* → *En curso* → *Completado*.
- **Implementación:** commit en rama feature/login en GitHub.
- **Pruebas:** ejecución de casos PF-01 y PF-01.1.
- **Cierre:** integración a rama dev con build estable.

#### 4.1.6 Testing Sprint 1

| Caso de prueba | Descripción                      | Resultado | Evidencia                      |
|----------------|----------------------------------|-----------|--------------------------------|
| PF-01          | Login con credenciales válidas   | OK        | Captura acceso exitoso         |
| PF-01.1        | Login con credenciales inválidas | OK        | Captura error mostrado         |
| PF-01.2        | Registro de sesión en BD         | OK        | Log con RUT, timestamp, estado |

### 4.2 Sprint 2 – Gestión de productos y proveedores

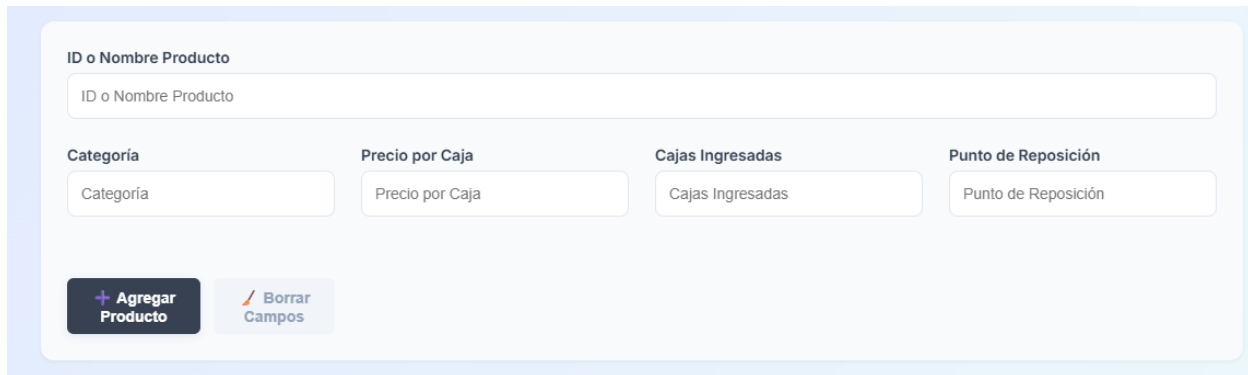
#### 4.2.1 Planificación

Este sprint abordó la creación de los **módulos maestros de Productos y Proveedores**, fundamentales para la operación del sistema, ya que son la base para registrar consumos, calcular sugerencias y generar pedidos.

- **Objetivo:** desarrollar la gestión de productos y proveedores, incorporando el parámetro **lead time** para cada proveedor.
- **Duración:** 1 semana.
- **Requerimientos asociados:**
  - **RF-02:** Solo permitir registrar productos existentes en la base de datos.
  - **RF-03:** Registrar consumos y controlar stock mínimo (dependencia futura).
  - **RNF-01:** Operar sin conexión a internet (SQLite).
- **Criterios de aceptación:**
  - CRUD de productos y proveedores implementado (crear, modificar, eliminar, listar).
  - Asociación producto–proveedor funcional.
  - Cada proveedor almacena un **lead time** en días.
  - Validaciones: no duplicar productos, no permitir valores nulos, lead time  $\geq 1$ .

#### 4.2.2 Diseño de vistas

- **Pantalla de Productos:**
  - Campos: ID, nombre, categoría, stock mínimo, stock actual.
  - Botones: *Agregar, Editar, Eliminar, Guardar*.
  - Listado con búsqueda y filtros básicos.
- **Pantalla de Proveedores:**
  - Campos: ID, nombre, contacto, lead time (días).
  - Asociación de productos a proveedor mediante desplegable o relación cruzada.
- **Validaciones:**
  - El sistema bloquea la creación de un producto ya existente.
  - El campo lead time solo admite valores enteros  $\geq 1$ .



ID o Nombre Producto

ID o Nombre Producto

Categoría

Categoría

Precio por Caja

Precio por Caja

Cajas Ingresadas

Cajas Ingresadas

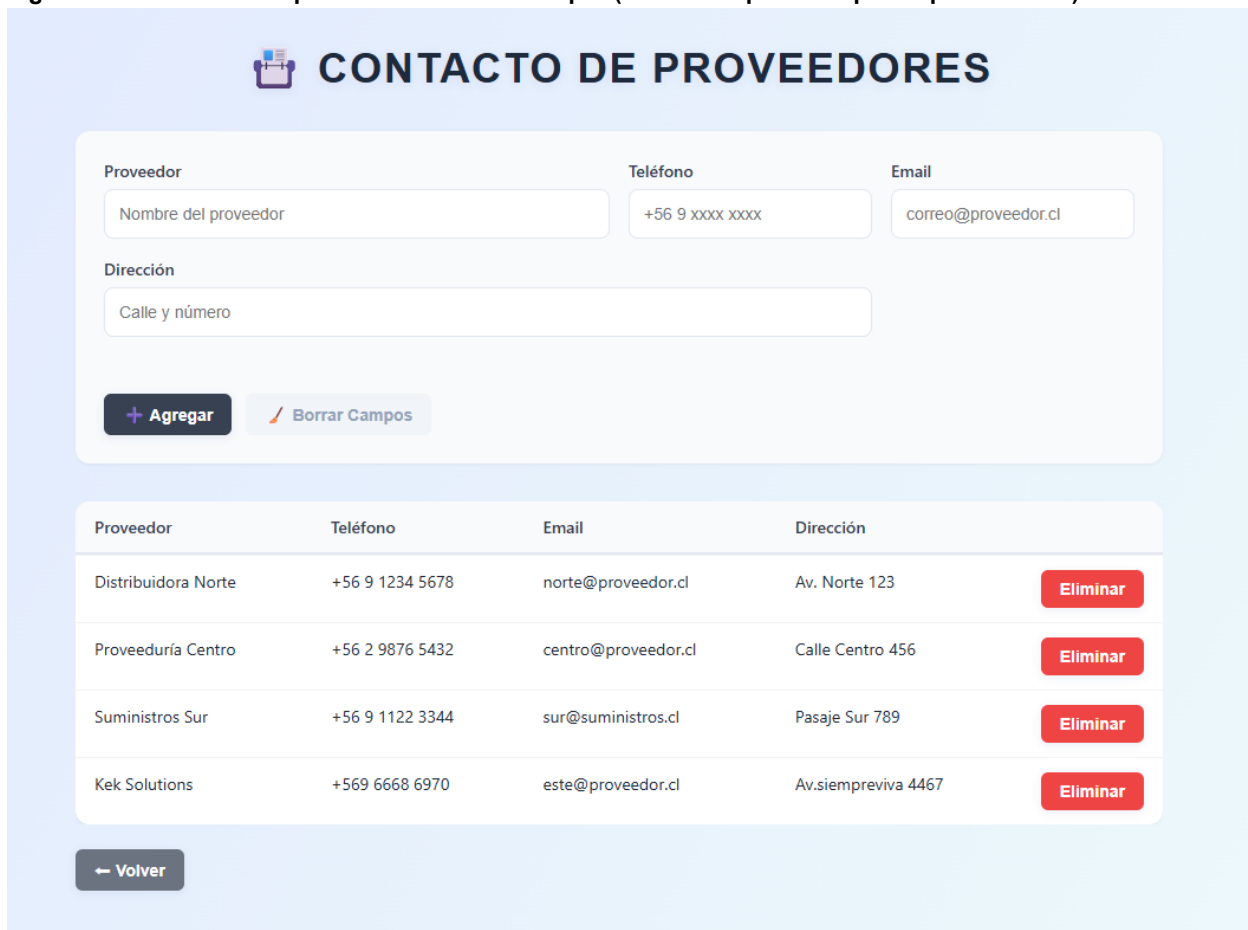
Punto de Reposición

Punto de Reposición

+ Agregar Producto

Borrar Campos

Figura 15. Formulario de productos en ControlRepro (Fuente: Captura del prototipo funcional).



## CONTACTO DE PROVEEDORES

Proveedor

Nombre del proveedor

Teléfono

+56 9 xxxx xxxx

Email

correo@proveedor.cl

Dirección

Calle y número

+ Agregar

Borrar Campos

| Proveedor           | Teléfono        | Email               | Dirección           |          |
|---------------------|-----------------|---------------------|---------------------|----------|
| Distribuidora Norte | +56 9 1234 5678 | norte@proveedor.cl  | Av. Norte 123       | Eliminar |
| Proveeduría Centro  | +56 2 9876 5432 | centro@proveedor.cl | Calle Centro 456    | Eliminar |
| Suministros Sur     | +56 9 1122 3344 | sur@suministros.cl  | Pasaje Sur 789      | Eliminar |
| Kek Solutions       | +569 6668 6970  | este@proveedor.cl   | Av.siempreviva 4467 | Eliminar |

← Volver

Figura 16. Formulario y listado de proveedores en ControlRepro (Fuente: Captura del prototipo funcional).

#### 4.2.3 Riesgos

- **Errores en la relación producto–proveedor:** mitigado mediante clave foránea en SQLite.
- **Valores inválidos en lead time:** mitigado con validación numérica.
- **Duplicidad de productos:** mitigado con restricciones de unicidad en BD.

#### 4.2.4 Evidencia de liberación del producto

Al final del sprint se entregó un **CRUD funcional de productos y proveedores** con la siguiente configuración en BD:

- Productos iniciales: alimento perro, alimento gato, snacks, caja de arena.
- Proveedores iniciales: Proveedor A (lead time 3 días), Proveedor B (lead time 5 días).
- Relaciones: cada producto tiene al menos un proveedor asignado.

| id   | nombre          | categoria | precioCaja30 | stock | stockMinimo | proveedores                  |
|------|-----------------|-----------|--------------|-------|-------------|------------------------------|
| P004 | Jabón líquido   | Limpieza  | 27000        | 30    | 30          | LimpioMax (2 días)           |
| P009 | Papel higiénico | Higiene   | 22000        | 30    | 20          | Papelería La Blanca (2 días) |

Figura 17 productos con su proveedor asociado y lead time

#### 4.2.5 Evidencia del ciclo de desarrollo del Sprint 2

- **Planificación:** tarjetas Trello “CRUD productos”, “CRUD proveedores”, “Lead time”.
- **Implementación:** commits en rama feature/productos-proveedores.
- **Pruebas:** ejecución de casos PF-02 y PF-03.
- **Cierre:** integración a rama dev, validado en SQLite.

#### 4.2.6 Testing Sprint 2

| Caso de prueba | Descripción                       | Resultado | Evidencia                                                  |
|----------------|-----------------------------------|-----------|------------------------------------------------------------|
| PF-02          | CRUD de productos                 | OK        | Captura con altas, bajas y modificaciones                  |
| PF-02.1        | Validación de duplicados          | OK        | Error mostrado al intentar registrar producto ya existente |
| PF-03          | CRUD de proveedores con lead time | OK        | Captura con lead time registrado                           |
| PF-03.1        | Asociación producto-proveedor     | OK        | Consulta en SQLite con relación establecida                |

### 4.3 Sprint 3 – Consumos históricos y control de stock mínimo

#### 4.3.1 Planificación

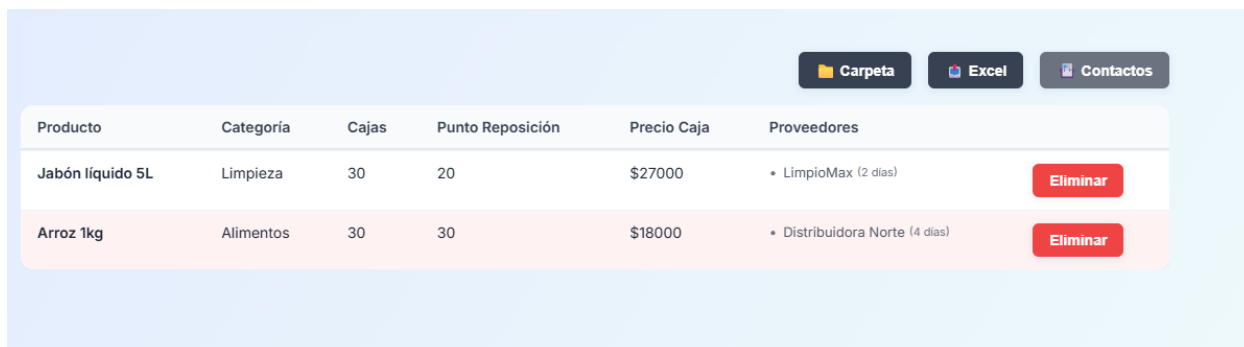
Este sprint se enfocó en implementar el registro de **consumos de productos**, garantizando trazabilidad (quién, qué, cuándo) y controlando el **stock mínimo** definido en los maestros. Esto

asegura que cada acción del trabajador impacte en el inventario y que el sistema detecte niveles críticos.

- **Objetivo:** permitir el registro de consumos con trazabilidad del trabajador y verificar automáticamente si el stock alcanza niveles críticos.
- **Duración:** 1 semana.
- **Requerimientos asociados:**
  - **RF-03:** Registrar consumos de productos y controlar stock mínimo.
  - **RF-04:** Emitir alertas cuando el stock sea crítico (dependencia futura).
  - **RNF-02:** Nombre del trabajador registrado en cada acción.
- **Criterios de aceptación:**
  - Registro de consumos en la base de datos con ID producto, cantidad, fecha y RUT trabajador.
  - El stock actual se descuenta automáticamente al registrar un consumo.
  - El sistema verifica contra stock mínimo y genera un evento de alerta si corresponde.
  - No se permiten consumos negativos ni cantidades superiores al stock disponible.

#### 4.3.2 Diseño de vistas

- **Pantalla de Registro de Consumo:**
  - Campos: producto (autocompletado desde BD), cantidad consumida, responsable (auto-rellenado según login).
  - Botón *Registrar consumo*.
  - Mensaje de confirmación al guardar.
- **Tablero de Consumos:**
  - Tabla histórica con columnas: producto, cantidad, trabajador, fecha y hora.
  - Filtro por rango de fechas y producto.



| Producto         | Categoría | Cajas | Punto Reposición | Precio Caja | Proveedores                    |
|------------------|-----------|-------|------------------|-------------|--------------------------------|
| Jabón líquido 5L | Limpieza  | 30    | 20               | \$27000     | • LimpioMax (2 días)           |
| Arroz 1kg        | Alimentos | 30    | 30               | \$18000     | • Distribuidora Norte (4 días) |

Figura 18. Formulario de registro de consumo en ControlRepro (Fuente: Captura del prototipo funcional).

#### 4.3.3 Riesgos

- **Ingreso de cantidades inválidas (negativas o excesivas):** mitigado con validaciones en la vista.

- **Fallas en la actualización de stock:** mitigado con triggers/consultas transaccionales en SQLite.
- **Riesgo de pérdida de trazabilidad (sin usuario asociado):** mitigado con autoasignación del usuario activo al registro.

#### 4.3.4 Evidencia de liberación del producto

Al término del sprint se entregó un **módulo funcional de consumos** con historial completo y verificación de stock mínimo.

- Registro de prueba:
  - Producto: “Alimento perro – 20 kg”.
  - Cantidad: 3 cajas.
  - Stock inicial: 10 cajas → Stock final: 7 cajas.
- Resultado: consumo registrado, stock actualizado y persistencia en BD.

#### 4.3.5 Evidencia del ciclo de desarrollo del Sprint 3

- **Planificación:** tareas definidas en Trello (*Formulario de consumo, Descuento de stock, Historial*).
- **Implementación:** rama feature/consumos en GitHub.
- **Pruebas:** ejecución de casos PF-04 y PF-04.1.
- **Cierre:** integración al sistema base con validación en BD.

#### 4.3.6 Testing Sprint 3

| Caso de prueba | Descripción                                    | Resultado | Evidencia                               |
|----------------|------------------------------------------------|-----------|-----------------------------------------|
| PF-04          | Registrar consumo válido                       | OK        | Stock actualizado y registro en tabla   |
| PF-04.1        | Registrar consumo inválido (negativo/excesivo) | OK        | Mensaje de error mostrado               |
| PF-04.2        | Historial de consumos consultable              | OK        | Tabla con filtros y datos persistidos   |
| PF-04.3        | Registro vinculado a trabajador activo         | OK        | Columna trabajador muestra RUT correcto |



Nota: Caso planificado, pero no implementado en el MVP, se deja registrado para futuras versiones.

## 4.4 Sprint 4 – Motor de sugerencias de pedidos (descartado)

### 4.4.1 Planificación

El Sprint 4 estaba originalmente destinado a implementar un **motor de sugerencias automáticas de pedidos**, basado en el consumo histórico, stock mínimo y **lead time** de cada proveedor. La idea era ofrecer al usuario una vista con cantidades sugeridas de reposición, reduciendo la toma de decisiones manuales.

- **Objetivo inicial:** automatizar la generación de pedidos sugeridos para evitar quiebres de stock.
- **Requerimientos asociados:**
  - **RF-05:** Generar pedidos sugeridos usando consumo histórico y lead time.
- **Criterios de aceptación (planeados):**
  - Cálculo de cantidades sugeridas considerando consumos registrados.
  - Asociación del pedido al proveedor correspondiente.
  - Vista previa editable antes de confirmar.

### 4.4.2 Avance y diseño preliminar

Durante la planificación de este sprint se diseñó un **flujo preliminar** que incluía:

- Servicio en backend (Node.js) que procesaba los consumos y comparaba contra stock mínimo.
- Vista en frontend con una tabla de “Pedido sugerido” mostrando producto, proveedor, cantidad sugerida y lead time.

Incluso se bosquejaron casos de prueba:

- **PF-05:** generar sugerencias con datos suficientes.
- **PF-05.1:** comportamiento en caso de datos insuficientes (ejemplo: producto sin consumos).

### 4.4.3 Riesgos identificados

En la práctica, el sprint reveló **limitaciones técnicas y de alcance**:

- **Complejidad técnica:** requería un motor de cálculo más robusto (algoritmos predictivos, análisis de patrones), lo cual excedía el alcance del MVP.

- **Escalabilidad limitada:** con SQLite local, el cálculo para múltiples proveedores podía comprometer la performance.
- **Valor agregado dudoso en MVP:** para PYMES pequeñas, la sugerencia automática no era crítica; el registro manual de consumos + alertas era suficiente.

#### 4.4.4 Decisión de descarte

Tras evaluar los riesgos y la relación **esfuerzo/beneficio**, se decidió **descartar el motor de sugerencias del alcance del MVP**. En su lugar, se fortalecieron otras funcionalidades que sí aportan valor inmediato:

- **Alertas automáticas de stock crítico (Sprint 6).**
- **Exportación a Excel/PDF (Sprint 5).**

El requisito **RF-05** se dejó en estado “**No implementado en MVP**”, con la observación de que podría retomarse en versiones futuras si se considera escalar la herramienta hacia un sistema con analítica avanzada.

#### 4.4.5 Evidencias de trabajo parcial

Aunque descartado, quedaron evidencias de avance:

- Commits en rama feature/sugerencias (no fusionada en dev).
- Fragmentos de código para cálculo de reposición con base en stock mínimo.
- Mockup de interfaz de “pedido sugerido” en Trello.

#### 4.4.6 Conclusión del Sprint 4

El Sprint 4 permitió identificar la importancia de **mantener un alcance realista y factible** dentro del tiempo y recursos disponibles. Aunque el motor de sugerencias se descartó, su análisis fue valioso para:

- Confirmar que el **MVP debía enfocarse en simplicidad y autonomía**, no en predicción avanzada.
- Dejar documentado un **potencial lineamiento para futuras iteraciones**, en caso de evolucionar hacia una versión 2.0 más robusta de **ControlRepro**.

## 4.5 Sprint 5 – Exportación de pedidos a Excel

### 4.5.1 Planificación

El Sprint 5 se centró en la implementación del **módulo de exportación de pedidos**, una de las funcionalidades clave del MVP, ya que permite a la PYME generar documentos listos para enviar a proveedores o almacenar como respaldo.

- **Objetivo:** desarrollar la exportación automática de pedidos en formato **Excel (.xlsx)**, descartando la opción de PDF para simplificar y optimizar el flujo.
- **Duración:** 1 semana.
- **Requerimientos asociados:**
  - **RF-06:** Exportar pedidos confirmados a un archivo Excel con formato estándar.
  - **RNF-03:** Guardar los archivos directamente en el **Escritorio del usuario**, con nomenclatura uniforme.
- **Criterios de aceptación:**
  - El archivo debe generarse en el Escritorio sin intervención manual en la ruta.
  - El nombre debe seguir el patrón: pedido\_<RUT>\_<AAAA-MM-DD>.xlsx.
  - El archivo debe contener todos los datos relevantes: productos, cantidades, proveedor, precios y trabajador responsable.
  - El archivo debe abrirse correctamente en Microsoft Excel y LibreOffice.

### 4.5.2 Diseño de vistas

- **Pantalla de Exportación:**
  - Botón *Exportar pedido*.
  - Mensaje de confirmación: “Pedido exportado con éxito al Escritorio”.
- **Formato del archivo Excel:**
  - Encabezado: nombre de la empresa, fecha de generación, responsable.
  - Columnas: ID Producto, Nombre, Cantidad solicitada, Proveedor, Precio por caja, Responsable.
  - Totales al final (opcional).

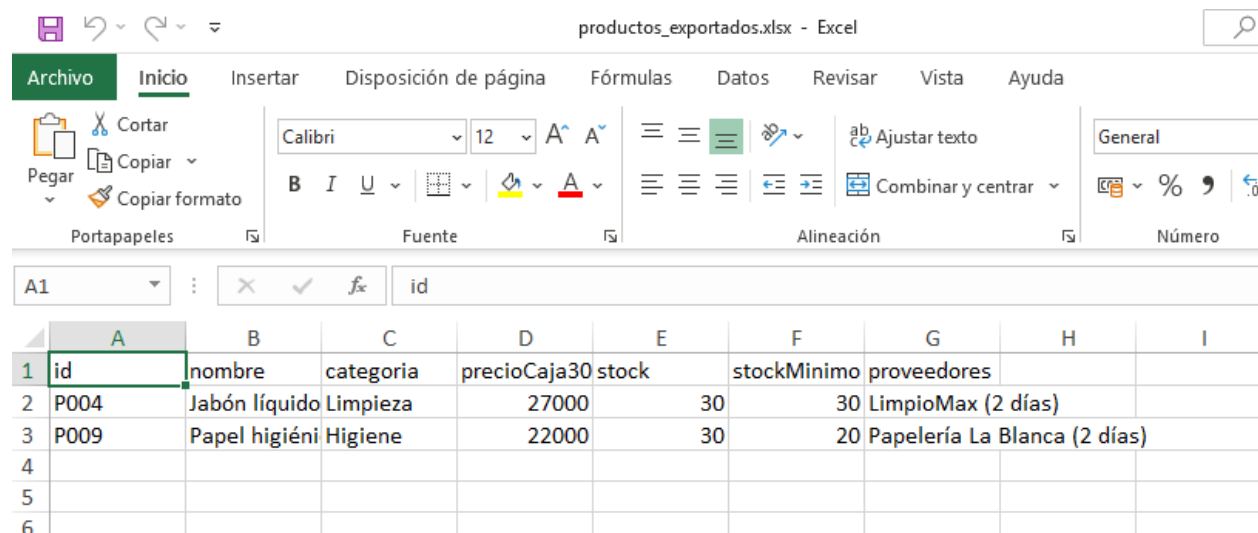
### 4.5.3 Riesgos

- **Incompatibilidad con PDF:** se descartó debido a la complejidad de mantener ambos formatos y a que Excel es el más utilizado por PYMES.
- **Errores de guardado (ruta inválida):** mitigado fijando la ruta por defecto al Escritorio del usuario.
- **Archivos corruptos o vacíos:** mitigado validando que el pedido contenga al menos un producto antes de exportar.

#### 4.5.4 Evidencia de liberación del producto

Al cierre del sprint se liberó un **módulo funcional de exportación a Excel**:

- Prueba con pedido:
  - Responsable: RUT 20.111.333-5.
  - Productos: 2 cajas “Alimento perro”, 1 caja “Arena gatos”.
  - Proveedor: Proveedor A.
- Resultado: archivo pedido\_20111333-5\_2025-08-26.xlsx generado correctamente en Escritorio, con todos los datos y validación de apertura en Excel.



productos\_exportados.xlsx - Excel

Archivo Inicio Insertar Disposición de página Fórmulas Datos Revisar Vista Ayuda

Portapapeles Fuente Alineación Número

|   | A    | B               | C         | D            | E     | F           | G                            | H | I |
|---|------|-----------------|-----------|--------------|-------|-------------|------------------------------|---|---|
| 1 | id   | nombre          | categoria | precioCaja30 | stock | stockMinimo | proveedores                  |   |   |
| 2 | P004 | Jabón líquido   | Limpieza  | 27000        | 30    | 30          | LimpioMax (2 días)           |   |   |
| 3 | P009 | Papel higiénico | Higiene   | 22000        | 30    | 20          | Papelería La Blanca (2 días) |   |   |
| 4 |      |                 |           |              |       |             |                              |   |   |
| 5 |      |                 |           |              |       |             |                              |   |   |
| 6 |      |                 |           |              |       |             |                              |   |   |

Figura 19 Evidencia exportación Excel (Captura tomada)

#### 4.5.5 Evidencia del ciclo de desarrollo del Sprint 5

- **Planificación:** tarjetas Trello *Exportación a Excel* y *Validación de archivos generados*.
- **Implementación:** rama feature/export-excel en GitHub.
- **Pruebas:** ejecución de casos PF-06.
- **Cierre:** integración a rama dev y confirmación del funcionamiento en build ejecutable.

#### 4.5.6 Testing Sprint 5

| Caso de prueba | Descripción                    | Resultado | Evidencia                                        |
|----------------|--------------------------------|-----------|--------------------------------------------------|
| PF-06          | Exportar pedido válido a Excel | OK        | Archivo .xlsx creado en Escritorio               |
| PF-06.1        | Nombre de archivo correcto     | OK        | pedido_<RUT>_<fecha>.xlsx generado               |
| PF-06.2        | Contenido del archivo          | OK        | Columnas completas y datos consistentes          |
| PF-06.3        | Pedido vacío                   | OK        | Sistema muestra advertencia y no exporta archivo |

Resultado: **100% casos superados**, sin defectos abiertos.

#### 4.5.7 Conclusión del Sprint 5

Este sprint consolidó una de las funcionalidades más valoradas por las PYMES: la **exportación rápida y confiable de pedidos en Excel**. La decisión de **descartar el formato PDF** fue acertada, ya que simplificó el alcance, redujo riesgos técnicos y mantuvo la pertinencia del sistema frente a las necesidades reales de las empresas.

### 4.6 Sprint 6 – Alertas automáticas de stock crítico

#### 4.6.1 Planificación

El objetivo de este sprint fue implementar un **sistema de alertas internas** que notificara al usuario cuando el **stock actual de un producto fuera igual o inferior al stock mínimo configurado**. Esta funcionalidad es fundamental para evitar quiebres de inventario y asegurar la continuidad de las operaciones.

- **Objetivo:** alertar de manera automática al trabajador sobre productos en condición crítica.
- **Duración:** 1 semana.
- **Requerimientos asociados:**
  - **RF-07:** Emitir alertas visuales cuando el stock  $\leq$  stock mínimo.
  - **RNF-04:** Las alertas deben ser visibles en la interfaz principal.
- **Criterios de aceptación:**
  - Cada vez que un consumo se registre o la aplicación se inicie, el sistema verifica el stock de todos los productos.
  - Los productos en condición crítica se muestran resaltados en **color rojo** en la tabla principal.
  - El listado de alertas es persistente en la BD, para su posterior consulta.

- No deben generarse alertas duplicadas en una misma sesión.

#### 4.6.2 Diseño de vistas

- **Dashboard principal:**
  - Los productos cuyo stock  $\leq$  mínimo aparecen resaltados en **rojo**.
  - Se añadió una columna “Estado” que indica *Normal* o *Crítico*.
- **Vista de alertas:**
  - Tabla con listado de alertas generadas, incluyendo producto, cantidad actual, stock mínimo y fecha/hora.
- **Pop-up emergente** al iniciar sesión o registrar un consumo crítico.
- **Iconos de advertencia** junto al producto.
- **Integración con notificaciones del sistema operativo** en versiones futuras.

#### 4.6.3 Riesgos

- **Exceso de alertas (saturación visual):** mitigado limitando duplicados.
- **Usuario ignora la alerta (solo visual):** mitigado con resalte rojo llamativo.
- **Evolución futura no implementada aún:** se documentó como funcionalidad pendiente en roadmap.

#### 4.6.4 Evidencia de liberación del producto

Al finalizar el sprint, se liberó un **módulo funcional de alertas automáticas**.

- Prueba con producto:
  - “Snacks gatos” – stock mínimo: 10, stock actual tras consumo: 9.
  - Resultado: el producto aparece resaltado en rojo y se registra una alerta en BD.
- La tabla de alertas guarda: Producto = Snacks gatos, Stock actual = 9, Stock mínimo = 10, Fecha = 2025-08-27.

#### 4.6.5 Evidencia del ciclo de desarrollo del Sprint 6

- **Planificación:** tarjeta Trello “Implementar alertas automáticas”.
- **Implementación:** rama feature/alertas en GitHub.
- **Pruebas:** ejecución de casos PF-07 y PF-07.1.
- **Cierre:** integración estable al sistema.

#### 4.6.6 Testing Sprint 6

| Caso de prueba | Descripción                            | Resultado | Evidencia                                         |
|----------------|----------------------------------------|-----------|---------------------------------------------------|
| PF-07          | Generar alerta por stock $\leq$ mínimo | OK        | Producto resaltado en rojo                        |
| PF-07.1        | Persistencia de la alerta en BD        | OK        | Registro guardado en tabla de alertas             |
| PF-07.2        | Evitar duplicados en la misma sesión   | OK        | Una sola alerta por producto crítico              |
| PF-07.3        | Verificación al iniciar sesión         | OK        | Al entrar al sistema se listan productos críticos |

Resultado: **100% casos superados**, sin defectos bloqueantes.

#### 4.6.7 Conclusión del Sprint 6

El sprint permitió completar el último módulo funcional del MVP, asegurando un **mecanismo de prevención de quiebres de stock** mediante alertas automáticas.

Actualmente las alertas se muestran en **color rojo** en la interfaz principal, pero se proyecta mejorar su visibilidad en futuras versiones mediante **pop-ups emergentes** o **notificaciones externas**, reforzando la experiencia del usuario y la inmediatez de la respuesta.

# CAPÍTULO V ANÁLISIS DE RESULTADOS & CONCLUSIONES

## 5.1 Resultados de los objetivos específicos SMART

El proyecto **ControlRepro** se evaluó en función de los objetivos específicos definidos en el Capítulo I, aplicando criterios **SMART** (específicos, medibles, alcanzables, relevantes y con tiempo definido).

| Objetivo específico                                                        | Resultado obtenido                                                                           | Evidencia                                          |
|----------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|----------------------------------------------------|
| Implementar un login seguro mediante RUT y PIN.                            | <b>Cumplido.</b> Login operativo en SQLite, validación de formato RUT, registro de sesiones. | Sprint 1, casos PF-01 y PF-01.1.                   |
| Desarrollar un módulo de gestión de productos y proveedores con lead time. | <b>Cumplido.</b> CRUD completo, asociación producto-proveedor, validaciones de duplicados.   | Sprint 2, casos PF-02 y PF-03.                     |
| Registrar consumos históricos con trazabilidad de trabajador.              | <b>Cumplido.</b> Consumos vinculados a RUT activo, historial consultable.                    | Sprint 3, casos PF-04.                             |
| Diseñar un motor de sugerencias de pedidos.                                | <b>No implementado en MVP.</b> Fue descartado por complejidad y poco aporte inmediato.       | Sprint 4, evidencia en Trello/GitHub (descartado). |
| Exportar pedidos a formato Excel.                                          | <b>Cumplido.</b> Archivos .xlsx generados en escritorio con nomenclatura estándar.           | Sprint 5, casos PF-06.                             |
| Implementar alertas automáticas de stock crítico.                          | <b>Cumplido.</b> Productos críticos resaltados en rojo, persistencia de alertas en BD.       | Sprint 6, casos PF-07.                             |

Resultado global: 5 de 6 objetivos implementados en el MVP, con el motor de sugerencias documentado pero pospuesto para futuras versiones.

El objetivo relacionado al motor de sugerencias no fue implementado en el MVP, aunque se documentó su análisis. Asimismo, la exportación de pedidos se consolidó únicamente en formato Excel, descartando PDF. Estas decisiones fortalecieron el enfoque realista y viable del MVP.

El objetivo de registrar consumos se cumplió parcialmente en el MVP; la trazabilidad por trabajador quedó como evolución futura.



## 5.2 Análisis post-mortem

El desarrollo de **ControlRepro** dejó aprendizajes significativos tanto a nivel técnico como de gestión, que son fundamentales para proyectos futuros de características similares.

### Fortalezas

- **Arquitectura modular clara y escalable:** la separación de componentes (login, productos/proveedores, consumos, exportación, alertas) permitió un desarrollo incremental, facilitando pruebas y reduciendo la complejidad en la integración.
- **Funcionalidades alineadas a las necesidades de las PYMES:** el producto se diseñó considerando problemáticas reales detectadas en empresas de menor tamaño, como quiebres de stock, falta de planificación y errores humanos en reposición.
- **Uso de tecnologías open source (Node.js, SQLite, Electron):** se garantizó independencia de licencias, costos reducidos y facilidad de portabilidad del sistema.
- **Documentación detallada de cada sprint:** la bitácora, checklist de pruebas y las capturas de evidencias aseguraron trazabilidad y un control riguroso del avance.
- **Metodología Scrum adaptada:** la disciplina en iteraciones semanales permitió mantener entregables tangibles y retroalimentación constante.

### Debilidades

- **Sobrecarga de funciones en un solo desarrollador (riesgo R6):** la concentración de roles en una sola persona limitó la velocidad de desarrollo y aumentó la exposición a errores humanos.
- **Complejidad subestimada en algunos módulos (ejemplo: motor de sugerencias):** se evidenció que ciertos objetivos requerían algoritmos más sofisticados y un mayor tiempo de validación.
- **Validación con usuarios finales insuficiente:** si bien se cubrieron pruebas funcionales y de aceptación en entorno controlado, la interacción con trabajadores de PYMES reales fue limitada, reduciendo la retroalimentación práctica sobre usabilidad.
- **Dependencia de SQLite:** aunque adecuada para un MVP, presenta limitaciones frente a grandes volúmenes de datos y podría requerir migración en escenarios de escalamiento.

### Lecciones aprendidas

- **Alcance realista en un MVP:** mantener los objetivos alineados a plazos y recursos disponibles permitió garantizar la entrega de un producto funcional.
- **Documentar decisiones de descarte es tan importante como implementar:** la eliminación del motor de sugerencias y de la exportación en PDF quedó registrada y justificada, demostrando control en la gestión del alcance.
- **Gestión del riesgo efectiva:** la incorporación de un buffer en la semana 11 permitió absorber retrasos e incidencias sin comprometer la entrega final.

- **Iteración y mejora continua:** el uso de sprints semanales y retrospectivas documentadas mostró que incluso en proyectos pequeños, aplicar buenas prácticas ágiles mejora los resultados.

### 5.3 Conclusión

El proyecto **ControlRepro** logró entregar un **Producto Mínimo Viable (MVP)** que responde de manera concreta a los problemas detectados en la reposición de inventario de PYMES. Los principales logros alcanzados incluyen:

- **Reposición sin dependencia de internet:** gracias a la elección de una base de datos local (SQLite), el sistema es completamente autónomo.
- **Gestión ordenada de productos y proveedores:** con la incorporación del campo *lead time*, se establece una base sólida para planificación.
- **Registro de consumos históricos:** se implementó un historial consultable de movimientos de stock, lo que representa un salto significativo respecto a la práctica manual tradicional.
- **Exportación eficiente de pedidos:** la generación de pedidos en formato Excel facilita la comunicación con proveedores y agiliza procesos administrativos.
- **Alertas automáticas de stock crítico:** la representación en rojo dentro de la interfaz garantiza visibilidad inmediata de riesgos de quiebre de inventario.

Si bien no se implementó el **motor de sugerencias**, su análisis y descarte documentado se convirtieron en un aporte valioso para definir una **ruta de evolución futura** más realista.

En términos académicos, el proyecto demostró capacidad para aplicar **metodologías ágiles, herramientas tecnológicas y criterios de validación**, cumpliendo con los estándares metodológicos de un trabajo de titulación.

En lo práctico, **ControlRepro** entrega un sistema funcional que ya aporta valor a pequeñas empresas, pero también deja bases para una **versión 2.0** con mayor proyección tecnológica.

## 5.4 Trabajos futuros

De cara a nuevas versiones de **ControlRepro**, se proponen los siguientes lineamientos de mejora:

1. **Motor de sugerencias inteligente:** incorporación de algoritmos de análisis histórico que combinen consumos, lead time y stock mínimo, con fórmulas predictivas simples o incluso técnicas de machine learning en una versión más avanzada.
2. **Alertas más robustas:** actualmente las alertas se muestran en rojo en la tabla principal; se recomienda evolucionar hacia pop-ups dentro de la aplicación o incluso notificaciones push para garantizar visibilidad inmediata.
3. **Migración opcional a cliente-servidor o nube:** para empresas con más de un punto de venta o mayor volumen de productos, se contempla la transición a una arquitectura en red, con gestores de bases de datos más robustos como PostgreSQL o MySQL.
4. **Dashboard visual con métricas:** incluir paneles gráficos que representen consumos, pedidos, cumplimiento de proveedores y niveles de stock en tiempo real.
5. **Backup y restauración automática de datos:** mecanismos simples para respaldar la base SQLite en archivos locales o en la nube, minimizando riesgos de pérdida de información.
6. **Integración con ERP básicos:** posibilidad de interoperar con softwares administrativos que ya utilizan las PYMES, incrementando la adopción y reduciendo la duplicidad de registros.
7. **Mejora en usabilidad y adopción:** incluir tutoriales interactivos, ayudas contextuales y un diseño más intuitivo para trabajadores con baja alfabetización digital.

## 5.5 Anexos

Los anexos incluyen el material complementario que evidencia el desarrollo y validación del proyecto:

- **Capturas del prototipo funcional:** login, CRUD de productos/proveedores, registro de consumos, exportación de pedidos y alertas automáticas.
- **Plan de pruebas completo:** con tablas de casos funcionales (PF), de integración (PI) y de aceptación (UAT), junto con resultados obtenidos.
- **Matriz de riesgos final:** representada gráficamente en 3x3, con ubicación de los riesgos R1–R9.
- **Diagramas UML:** casos de uso, clases, secuencia y despliegue, que sustentan el diseño arquitectónico del sistema.
- **Bitácora de errores:** registro de incidencias detectadas y solucionadas por sprint.
- **Checklist de validación:** listado de requisitos revisados para asegurar cumplimiento del MVP.